الجامعة المصرية اليابانية للعلوم و التكنولوجيا
エジプト日本科学技術大学
EGYPT-JAPAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Sustainable Edge Intelligence: A Unified Reinforcement Learning Framework for Energy-Harvesting Vision Systems

CSE500: Graduation Project 2

Name	**University ID**
Ayatollah Ibrahim Abdellatif Elkolally	120210312
Anas Osama Ali Attya Dorgham	120210156

Supervised by

Dr. Rami Zewail
Prof. Mohammed S. Sayed
Prof. Inoue Koji

Contents

Abstract

The deployment of intelligent vision systems on resource-constrained, battery-less edge devices remains a significant challenge due to the stochastic nature of ambient energy harvesting. While advances in Tiny Machine Learning (TinyML) have enabled the migration of deep learning models to microcontrollers, the unpredictable nature of energy sources often results in system failures or the adoption of overly conservative models that sacrifice accuracy. To resolve this, a unified decision-making framework is introduced to jointly optimize sensing frequency, inference complexity through multi-model selection, and communication strategies.

The system is formulated as a discrete-time Markov Decision Process (MDP) to capture the complex interdependencies between energy generation, battery storage, and vision-based workloads. A Proximal Policy Optimization (PPO) reinforcement learning agent serves as the core controller, trained using a six-stage curriculum learning strategy that progressively increases task difficulty. To ensure long-term sustainability and prevent catastrophic energy depletion, a soft safety shield is integrated to intervene when battery levels reach critical thresholds.

Experimental evaluation conducted on NVIDIA Jetson Nano and Raspberry Pi 3 platforms demonstrates that the reinforcement learning agent significantly outperforms traditional heuristic and Model Predictive Control (MPC) baselines. In expert-level scenarios, the framework achieved a 74.6 percent mean event delivery rate and maintained 100 percent episode completion when equipped with the safety mechanism. Furthermore, cross-platform transfer learning experiments revealed that pre-training on a compute-rich device can reduce the required training episodes on a more constrained platform by approximately 90 percent. This research provides a scalable and robust path for deploying sustainable, high-fidelity AI in autonomous, deploy-and-forget environmental monitoring systems.

Chapter 1

Introduction

The rapid convergence of Artificial Intelligence (AI) and Internet of Things (IoT) technologies is reshaping the landscape of distributed computing, enabling intelligence to migrate from centralized cloud infrastructures toward resource-constrained edge devices. This transition is driven by the growing demand for real-time responsiveness, reduced communication latency, enhanced privacy, and scalable deployment across large sensor networks.

Despite these advances, deploying sophisticated machine learning models on energy-constrained embedded platforms remains a fundamental challenge. Edge devices must operate within strict limitations on memory, computational capacity, and power availability, often while supporting continuous sensing and autonomous decision-making. These constraints become significantly more pronounced in battery-less systems powered by ambient energy harvesting, where energy availability is inherently stochastic and unpredictable.

This chapter reviews the technological foundations and research developments that underpin intelligent energy-harvesting edge systems. It begins by examining the evolution from Cloud AI to Edge AI and the emergence of Tiny Machine Learning (TinyML) as a key enabler of on-device intelligence. It then analyzes the primary barriers limiting sustainable deployment, namely hardware constraints, intermittent energy supply, and the trade-off between inference accuracy and energy consumption.

Subsequently, the chapter explores reinforcement learning as a promising paradigm for adaptive system control under uncertainty. By synthesizing insights from prior work, the discussion highlights the fragmentation of existing approaches and motivates the need for a unified decision-making framework capable of jointly optimizing sensing, inference, and communication. The chapter concludes by identifying the research gap addressed in this thesis and outlining the core problem it seeks to solve.

1.1 Evolution of Edge Intelligence: Intelligent Edge Devices

The paradigm of Artificial Intelligence (AI) is undergoing a transformative shift from Cloud AI to Edge AI. While traditionally deep learning has relied on high-performance cloud servers to handle its massive computational and memory demands, this approach is increasingly limited by significant network latency, high bandwidth costs, and growing data privacy concerns [16]. Edge computing has emerged to address these hurdles by decentralizing computation, allowing data to be processed near the point of capture. This transition enables real-time responsiveness and autonomous decision-making directly on the device, which is essential for applications such as personalized healthcare, smart manufacturing, and autonomous vehicles [24].

Recent advances in Tiny Machine Learning (TinyML) have further pushed these boundaries, enabling the deployment of deep learning models directly on microcontrollers (MCUs), thereby expanding AI capabilities to billions of always-on IoT devices [16, 24]. TinyML enables localized intelligence using only a few hundred kilobytes of memory, which significantly reduces the cost of deployment and the energy expenditure associated with constant wireless communication [30]. By performing on-device inference near the sensor, TinyML transforms simple IoT nodes into intelligent compute nodes capable of sophisticated data analytics [16].

However, the realization of TinyML is constrained by extreme hardware and memory limitations. Microcontroller units typically possess on-chip memory (SRAM) and storage (Flash) resources that are 2 to 3 orders of magnitude smaller than those of mobile phones and up to 6 orders of magnitude smaller than cloud-based GPUs [14]. For instance, a state-of-the-art ARM Cortex-M7 MCU may only provide 320 KB of SRAM and 1 MB of Flash storage. Off-the-shelf deep learning models often exceed these limits by significant margins: a standard ResNet-50 model exceeds MCU storage limits by 100×, and even int8 quantized versions of MobileNetV2 still exceed the memory budget by over 5× [14].

Critically, the primary bottleneck in TinyML is activation memory, rather than just the number of model parameters. Traditional deep learning models designed for mobile platforms (such as ShuffleNet or standard MobileNets) focus on reducing parameters and FLOPs but often disregard peak memory usage. This mismatch means that directly scaling down mobile-class models is often insufficient for the bare-metal, resource-scarce environment of microcontrollers, which lack the support of an operating system or external DRAM [14]. Consequently, there is an urgent need for system-algorithm co-design to ensure that efficient neural architectures are tailored specifically to the unique memory and power envelopes of tiny edge hardware [24].

1.2 The Intermittency Wall: Unveiling the Real Bottleneck

While the memory and compute limitations of microcontrollers present a formidable barrier, the ultimate bottleneck for the next generation of Edge AI lies in the energy source itself. The growing demand for sustainable, "deploy-and-forget" systems has led to a radical transition away from traditional batteries, which are bulky, environmentally hazardous, and require expensive manual replacement. In their place, Zero-Energy Devices (ZEDs) or battery-less IoT nodes have emerged, designed to rely exclusively on energy scavenged from ambient sources such as solar, thermal gradients, or kinetic vibrations [30].

However, this shift introduces a fundamental conflict: running TinyML on battery-less devices is extremely challenging due to the unpredictable and dynamic harvesting environment. Unlike cloud or mobile platforms that benefit from stable power grids, battery-less sensors operate in a state of constant energy flux. Ambient sources rarely provide constant power, and the energy harvested is often insufficient to run a device continually, frequently delivering only scant microwatts of power [31].

This volatility forces the device into a paradigm of intermittent computing. The system must painstakingly accumulate energy in a small buffer, such as a capacitor, only to consume it in a brief burst of activity before the power is abruptly exhausted. These recurring power failures are catastrophic for traditional AI logic; they erase the system's volatile memory and system state, potentially trapping the inference process in a loop of "non-termination" where the program gets stuck and never completes its task [2, 21].

The tension reaches a breaking point when considering that standard deep neural networks (DNNs) are inherently rigid. They are trained for a world of energy abundance and assume that every input requires the same, fixed computational path. In the erratic world of a battery-less sensor, a fixed inference strategy is not just inefficient—it is infeasible. Without a mechanism to react to the instantaneous energy envelope, these "intelligent" nodes are rendered useless during energy shortfalls, failing to meet even basic service level objectives and risking total system blackout [22, 31].

1.3 The Accuracy Wall: The Paradox of Worst-Case Design

The fundamental challenge in deploying intelligence on Zero-Energy Devices (ZEDs) is the conflict between model fidelity and energy sustainability. Current design paradigms typically resort to static model compression—such as aggressive pruning and quanti-

zation—to ensure that the system remains operational under the most stringent power conditions [12, 31]. However, this approach introduces a significant and often overlooked trade-off: models are frequently compressed to fit worst-case energy constraints, which leads to a permanent and unnecessary reduction in accuracy [4, 11].

This "lowest common denominator" design philosophy is inherently inefficient. When environmental energy is plentiful (e.g., peak solar intensity), a statically compressed model continues to provide low-accuracy results, wasting the surplus energy that could have powered a more sophisticated inference [22]. Conversely, if a designer opts for a high-fidelity model, the device risks frequent power failures and "non-termination" loops during energy shortfalls, rendering the system unreliable [21, 31]. Existing research indicates that this rigidity is the primary barrier to achieving human-level perception at the extreme edge [11].

This tension highlights the critical need for the core components of this thesis: adaptive inference, intelligent scheduling, and real-time decision-making. Rather than relying on a single, rigid network, an intelligent system must be capable of multi-model selection (MMS) or early exiting (EE) to dynamically adjust its computational intensity based on the instantaneous energy storage state and the harvesting rate [4, 12, 31].

By implementing an energy-aware scheduler, a device can "up-shift" to a heavyweight model when energy is abundant to maximize Quality of Service (QoS), and "down-shift" to a lightweight, compressed version during energy scarcity to maintain forward progress [13, 22]. This transition from static budgets to dynamic, energy-reactive intelligence is the only viable path to breaking the accuracy wall and enabling sustainable, high-performance Edge AI [4, 11].

1.4 The Feasibility Breakthrough: Tiny AI in Practice

Despite the formidable constraints of the edge, recent research has demonstrated that deep learning is no longer the exclusive domain of high-performance servers. The advent of Tiny Machine Learning (TinyML) has proven that it is possible to "squeeze" sophisticated neural networks into microcontrollers (MCUs) that previously seemed incapable of such tasks [14, 24].

The magnitude of this challenge cannot be overstated: microcontrollers have memory 2–3 orders of magnitude smaller than mobile phones, making deployment difficult. While a mobile phone may have gigabytes of DRAM, a state-of-the-art ARM Cortex-M7 MCU provides only 320 KB of SRAM and 1 MB of Flash storage [14]. Standard deep learning models designed for mobile platforms, such as ResNet-50 and MobileNetV2, exceed these storage and peak memory limits by up to 100× and 22× re-

spectively. Even aggressively quantized versions of these models often fail to fit on-chip, illustrating a massive gap between desired hardware capacity and available resources [14].

However, the landmark framework MCUNet has shown that this barrier can be overcome through system-algorithm co-design [14]. By jointly optimizing the neural architecture (TinyNAS) and a specialized inference engine (TinyEngine), MCUNet achieved a record 70.7% ImageNet top-1 accuracy on a commercial microcontroller—the first time ImageNet-scale inference has been realized at such a tiny scale. This breakthrough was made possible by shifting operations from runtime to compile-time, eliminating the memory overhead of traditional interpreters, and using patch-based inference to break the memory bottleneck of initial layers [14].

This realization transitions the field to a new, even more critical problem: If AI can run on tiny devices through architectural optimization, how should it be scheduled under energy uncertainty? While MCUNet proves that the "computational wall" can be breached, these optimized models remain static and rigid. In a battery-less environment where energy is harvested intermittently, even a highly optimized model may be too "heavy" for a low-energy slot, or too "light" to take advantage of a power surplus [31]. This emerging research question forms the core of this thesis: finding a mechanism to dynamically manage these now-feasible models in response to a stochastic and unpredictable power budget.

1.5 Energy Harvesting Systems and the Shift toward Reinforcement Learning

As the Internet of Things (IoT) expands toward billions of connected devices, traditional battery-powered sensing is becoming a major bottleneck due to finite lifespans and the high maintenance costs of manual replacement. To achieve true operational longevity, next-generation IoT nodes increasingly rely on energy harvesting (EH) from ambient sources like solar radiation, thermal gradients, and kinetic vibrations. While EH offers a path to self-sustained, "deploy-and-forget" systems, the captured energy is often erratic, delivering only scant microwatts of power that fluctuate based on environmental conditions [9, 10].

The primary challenge in managing these devices is the requirement for intelligent configuration that matches the application's sensing needs to the instantaneous energy envelope. Historically, configuration has relied on manual tuning or ad-hoc heuristics, both of which require significant domain expertise and fail to scale as deployments move from single devices to massive, heterogeneous sensor networks. Because ambient energy availability is stochastic and environment-dependent—such as indoor light being

influenced by human occupancy—static rules often lead to either wasted surplus energy or catastrophic system failure due to energy depletion [9, 26].

To address these limitations, recent research has transitioned toward Reinforcement Learning (RL) as a transformative mechanism for autonomous sensor management. As highlighted in the landmark work *"Scaling Configuration of Energy Harvesting Sensors with Reinforcement Learning"* [9], RL represents a paradigm shift where the device no longer relies on pre-programmed rules. Instead, an RL agent interacts with its specific environment to learn energy availability patterns and dynamically configure parameters like sensor sampling rates. By observing states such as the current capacitor voltage and environmental energy income, the agent learns to optimize a reward function that balances two conflicting goals: maximizing sensing quality and preventing energy depletion [10].

This RL-based approach is particularly effective because it can identify complex diurnal patterns and human-induced energy fluctuations that are invisible to static heuristics. Using algorithms like Q-learning, the system can "up-shift" its duty cycle when energy is abundant to maximize Quality of Service (QoS) and "down-shift" during energy shortfalls to maintain forward progress. Furthermore, to make these systems practical for real-world deployment, techniques such as transfer learning and day-by-day training have been developed to reduce the initial "exploration" phase, allowing battery-less nodes to become operational and energy-neutral within their first day of deployment [10]. This synergy between adaptive learning and ambient harvesting forms the necessary foundation for the sustainable, intelligent edge systems explored in this thesis.

1.6 Reinforcement Learning for Complex Energy-Aware Decision Making

As the complexity of Edge AI systems increases, traditional heuristic and rule-based methods reach their limits. Managing an energy-harvesting system is not merely a task of reducing consumption; it is a sequential decision-making problem that must account for the stochastic and time-varying nature of ambient energy. Decisions made in the current time step—such as selecting a high-fidelity model or increasing the sampling rate—directly impact the energy reserves available for future operations [13, 38].

To address these challenges, researchers have transitioned to the Markov Decision Process (MDP) framework, which provides a structured mathematical foundation for modeling long-term optimization under uncertainty. In this paradigm, a Reinforcement Learning (RL) agent interacts with the environment to learn optimal policies that maximize a long-term cumulative reward, typically balancing performance (such as

accuracy or data freshness) against the risk of energy depletion [3, 13].

A prominent example of this is found in recent work on Meta-Reinforcement Learning for Timely and Energy-efficient Data Collection in Solar-powered UAV-assisted IoT Networks [38]. In this study, the problem is explicitly formulated as an MDP to jointly optimize scheduling and energy consumption. The RL agent must coordinate multiple conflicting objectives: minimizing the Age of Information (AoI) from sensor nodes, managing the UAV's propulsion energy, and adapting to unpredictable solar energy arrival. By utilizing meta-learning, the system extracts generalized knowledge from various tasks, allowing the device to converge quickly to an optimal policy even when environmental conditions shift [38].

Furthermore, RL-based controllers have demonstrated the ability to identify complex diurnal patterns and human-induced energy fluctuations that are invisible to static heuristics. For instance, in smart building sensors, RL agents successfully learned to "up-shift" sensing frequency during peak light availability and "down-shift" during periods of scarcity to maintain energy neutrality [10]. By leveraging the Bellman optimality principle, RL ensures that inference and communication decisions are not just locally efficient but globally sustainable across a long-term operational horizon [3, 25]. This capability confirms that RL is the most appropriate mechanism for orchestrating the sophisticated, multi-objective trade-offs required by modern energy-harvesting Edge AI systems.

Collectively, these studies demonstrate substantial progress toward enabling intelligence on resource-constrained platforms. However, the literature remains largely fragmented, with most approaches optimizing individual subsystems rather than addressing the coupled nature of sensing, inference, and communication. This fragmentation motivates the need for holistic control strategies capable of coordinating multiple decision variables under stochastic energy conditions.

1.7 Research Gap

Recent advancements in Tiny Machine Learning (TinyML) have successfully facilitated the deployment of deep neural networks on highly resource-constrained embedded platforms, enabling intelligent sensing directly at the extreme network edge. Simultaneously, energy-harvesting (EH) technologies have emerged as a sustainable solution for long-term autonomous operation of Internet of Things (IoT) devices by scavenging ambient power from solar, thermal, or kinetic sources. Despite these developments, the reliable operation of battery-less intelligent edge devices remains fundamentally obstructed by the intermittent and stochastic nature of harvested energy.

Prior research has extensively explored energy-aware system design, but primarily through the lens of isolated optimization objectives. For instance, current literature

commonly addresses individual problems such as adaptive sensor sampling [10], dynamic voltage scaling [26], workload offloading [7, 36], or lightweight model selection [12, 31] to minimize energy footprints. While these isolated approaches improve specific efficiencies, they typically treat sensing, inference, and communication as independent processes rather than tightly coupled decision variables within a unified control framework.

Real-world intelligent vision systems operate under multi-dimensional trade-offs that simple isolated optimizations cannot resolve. Increasing sensing frequency improves event detection but significantly heightens the risk of energy depletion and system failure. Selecting high-fidelity inference models enhances predictive accuracy but incurs quadratic computational and memory costs. Furthermore, transmitting raw data ensures maximum fidelity at the host but consumes prohibitive communication energy compared to local processing [20, 36]. These interdependencies create a complex sequential decision-making problem that cannot be effectively solved using static policies.

Moreover, existing methods frequently rely on heuristic scheduling rules or worst-case provisioning, which often lead to overly conservative behavior and the underutilization of harvested energy during periods of peak availability [4, 22]. Such strategies lack the adaptability required for dynamic environments where energy income can fluctuate by several orders of magnitude. While reinforcement learning (RL) has been proposed to manage energy-harvesting sensors [3, 10], prior studies largely focus on narrow objectives like duty-cycle adjustment or transmission scheduling [25, 38]. Limited attention has been given to the joint optimization of sensing, adaptive inference (via multi-model selection or early exiting), and communication, particularly in vision-based edge platforms where resource demands are significantly higher than traditional scalar sensors.

Therefore, a critical gap remains in the design of intelligent control mechanisms capable of:

- Continuously adapting device behavior to the stochastic and unpredictable availability of ambient energy.
- Balancing inference accuracy against energy expenditure in real-time through dynamic architectural adjustments.
- Coordinating sensing, processing, and transmission decisions within a unified, hierarchical framework.
- Maximizing long-term system utility without compromising the operational safety and forward progress of the device.

Addressing this gap requires a unified decision-making framework, potentially based on Markov Decision Processes (MDPs), that can learn optimal policies directly from environmental interactions while accounting for the complex dynamics of energy harvesting and vision-based workloads.

1.8 Problem Statement

Motivated by the limitations of current static and isolated optimization strategies, this thesis investigates the following core research question:

> *How can an energy-harvesting vision device dynamically adapt its sensing, inference, and communication strategies to maximize task performance while ensuring sustainable operation under uncertain energy conditions?*

1.9 Thesis Contributions

This thesis presents a reinforcement-learning-based adaptive inference framework designed to enable sustainable and intelligent operation of energy-harvesting vision devices. Unlike conventional approaches that optimize isolated subsystems, the proposed framework jointly coordinates sensing, inference, and communication decisions within a unified control architecture.

The main contributions of this work are summarized as follows:

1. Formal System Modeling: A comprehensive mathematical model of an energy-harvesting vision system is developed, capturing the stochastic dynamics of energy generation, battery storage, workload arrivals, buffering behavior, and task-dependent energy consumption. The model provides a principled foundation for sequential decision-making and policy optimization.

2. Unified Markov Decision Process Formulation: The device operation is formulated as a discrete-time Markov Decision Process (MDP) that integrates energy state, workload conditions, and system resources into a structured state representation. This formulation enables the learning of adaptive control policies that optimize long-term performance rather than short-term heuristics.

3. Adaptive Inference and Scheduling Framework: A novel control strategy is proposed in which the device dynamically selects among multiple operational modes, including sensing rates, inference complexities, and transmission options. By explicitly modeling the tradeoff between accuracy and energy expenditure, the framework allows the system to intelligently scale computational effort according to available resources.

4. Reinforcement Learning-Based Policy Optimization: Deep reinforcement learning is employed to learn energy-aware policies directly from interaction with the simulated environment. The approach eliminates the need for manually tuned scheduling rules and enables the system to adapt to diverse and time-varying harvesting conditions.

5. Safety-Aware Operational Constraints: To promote sustainable operation, the framework incorporates mechanisms that discourage unsafe behaviors such as aggressive energy consumption leading to battery depletion. This ensures reliable long-term performance while maintaining responsiveness to workload demands.

6. Extensive Experimental Evaluation: A large-scale experimental study is conducted across thousands of simulated episodes and multiple policy configurations to evaluate system behavior under varying environmental conditions. Statistical significance testing is performed to rigorously validate performance improvements over baseline strategies.

7. Transferability and Practical Deployment Insights: The study further investigates the potential for policy transfer across heterogeneous energy environments, demonstrating that previously learned behaviors can substantially reduce retraining requirements. These findings provide practical insights for real-world deployment of intelligent energy-harvesting devices.

Chapter 2

Literature Review

2.1 TinyML Systems

Tiny Machine Learning (TinyML) is an emerging paradigm defined by the deployment of machine learning and deep learning models onto highly resource-constrained embedded hardware, specifically microcontroller units (MCUs) [16, 34]. Unlike traditional cloud-based AI, TinyML systems operate under extreme constraints, typically within a power budget of less than 1 mW and memory footprints ranging from 32 KB to 512 KB of SRAM [34]. These systems are critical for the advancement of edge devices and the Internet of Things (IoT), as they enable real-time, on-device data processing that significantly reduces latency, conserves bandwidth, and enhances data privacy by eliminating the need to transmit sensitive sensor data to external servers [14, 34]. Key frameworks have pioneered this field by utilizing system-algorithm co-design; most notably, *MCUNet* achieved ImageNet-scale accuracy (over 70%) on off-the-shelf microcontrollers with only 320 KB of SRAM [14]. This was made possible through *TinyNAS*, a two-stage neural architecture search that automatically optimizes model structures for specific hardware constraints, and *TinyEngine*, a memory-efficient inference engine that optimizes memory scheduling based on the overall network topology [14].

The design of these systems is governed by a fundamental tension between model representational power and hardware limits. Microcontrollers lack dedicated GPUs and often lack floating-point units (FPUs), forcing models to rely on integer arithmetic and lower clock speeds [14, 34]. Memory serves as the primary bottleneck, categorized into Flash storage for model parameters (weights) and SRAM for peak activations and intermediate buffers [1, 14]. To fit within these envelopes, designers employ aggressive techniques such as quantization to INT8 or lower, pruning of redundant connections, and patch-based inference, which executes feature maps in tiles to avoid overusing the limited SRAM buffer [15, 34, 39]. Furthermore, algorithm-hardware co-design ensures that the neural architecture is specialized for the target MCU's specific memory

hierarchy and compute capabilities [14].

Despite their success in enabling feasibility, most current TinyML solutions are limited by a static approach to power consumption. A critical assessment of these systems reveals that they largely assume a stable power supply and do not dynamically adapt to the intermittent nature of energy-harvesting (EH) environments [19, 29]. While these methods achieve remarkable accuracy on battery-powered IoT devices, they are often insufficient for battery-less, stochastic EH devices where energy availability fluctuates unpredictably [29]. For instance, a fixed inference strategy that fits a 256 KB memory envelope may still fail if the device experiences a power failure during execution due to a depleted energy buffer [29]. In summary, while current methods effectively enable complex inference on restricted hardware, none are designed to adapt dynamically to stochastic energy conditions.

2.2 Energy-Aware ML on Edge Devices

The realization of energy-aware (EA) inference at the extreme edge requires navigating a complex multi-dimensional tradeoff between classification accuracy, latency, and energy consumption [5, 6]. Energy is a critical constraint for TinyML systems because the energy consumption of a Deep Neural Network (DNN) is not always directly proportional to its memory or computational footprint; factors such as data movement can account for up to 90% of a model's energy usage [19]. Traditionally, many edge AI systems have relied on a "worst-case" design paradigm, where models are excessively compressed to fit the most stringent energy envelopes. However, this approach is fundamentally limited as it sacrifices significant accuracy even when harvestable ambient energy is plentiful [19, 32].

To address these limitations, recent research has shifted toward adaptive inference designs that dynamically adjust model complexity based on real-time energy availability. Two prominent techniques include multi-model selection (MMS) and early exiting (EE) [5, 6]. MMS frameworks, such as those proposed by Sabovic et al., deploy a subset of alternative TinyML models concurrently and dynamically select the most accurate version that fits within current memory and energy constraints [32]. Conversely, early-exit networks like *ePerceptive* incorporate internal classifiers at intermediate layers, allowing samples to terminate early if sufficient confidence is reached, thereby bypassing unnecessary subsequent layers to save energy [5, 22]. A related scheduling challenge involves the decision between local vs. cloud inference. Local inference provides low latency, improved data privacy, and predictable response times by keeping data on-device [19, 30]. Cloud-based inference, while capable of running heavy-weight models with higher accuracy, often incurs prohibitive energy and bandwidth costs due to the need for continuous data transmission over unpredictable wireless channels [19, 30].

Despite these advancements, a critical assessment of existing literature reveals that static model selection remains highly wasteful during periods of energy surplus, as it fails to maximize performance when resources are abundant [32]. While adaptive frameworks like *ePerceptive* react to energy fluctuations, many are heuristic in nature and lack the long-term theoretical grounding required to handle truly intermittent environments [5, 22]. While the field has explored various forms of adaptive inference, none fully handle joint adaptive inference and stochastic energy harvesting.

2.3 Energy Harvesting IoT Optimization

Energy harvesting (EH) Internet of Things (IoT) systems aim for perpetual operation by scavenging energy from ambient sources such as solar, thermal, and mechanical vibration [2, 10, 26, 28]. However, the primary research constraint in these systems is the stochastic and unpredictable nature of ambient energy [5, 10, 26]. Harvestable power varies drastically depending on the time of day, meteorological conditions, and human activity; for instance, solar power availability can fluctuate by up to three orders of magnitude between sunny and shadowy environments [9, 10, 26]. Unlike battery-powered devices with stable energy, EH-powered nodes face frequent power failures and shutdowns when the harvested income falls below the minimum operating threshold [2, 26, 28]. This necessitates intermittent computing, a paradigm where the system performs computation in bursts only when sufficient energy is available, relying on checkpoints or non-volatile memory to preserve forward progress across power cycles [2, 21, 28].

Existing solutions to manage this variability traditionally rely on heuristic or static duty cycles, which often undersize performance to accommodate worst-case energy scenarios [10, 26]. To move beyond these static limitations, researchers have proposed dynamic optimization models. For example, *Rao et al.* utilize a Markov Decision Process (MDP) to derive optimal task scheduling policies that account for task priorities, deadlines, and the stochastic arrival of harvested energy [26]. Other approaches employ Reinforcement Learning (RL), such as the *ACES* framework by *Fraternali et al.*, to automatically configure sensor duty cycles and maximize utility in fluctuating environments [10]. Furthermore, specialized hardware and software techniques like *MOUSE* utilize non-volatile memory to enable efficient DNN inference by reducing the overhead of intermittent restarts [28].

Despite these advancements, a critical assessment reveals that most existing EH optimization research focuses predominantly on sensor-level duty cycles rather than complex processing [10, 26]. Prior works rarely consider vision inference pipelines or the complex requirements of joint sensing, inference, and communication tasks [5, 21]. While these models effectively manage low-power sensing, they often struggle with the

significant and varying energy demands of Deep Neural Networks (DNNs) [5, 21, 28]. Consequently, adaptive policies are strictly required to handle the high variability of energy harvesting and ensure the completion of complex tasks within an unpredictable energy envelope [21, 26].

2.4 Reinforcement Learning for Resource Management

Reinforcement Learning (RL) has emerged as the natural paradigm for adaptive decision-making in energy-harvesting (EH) systems because the ambient energy supply is inherently stochastic and unpredictable. These systems require a series of sequential decisions over time to manage energy buffers, a requirement that Markov Decision Processes (MDPs) are mathematically designed to address. RL agents learn optimal policies through trial-and-error interactions with the environment, which is essential for achieving Energy Neutral Operation (ENO) while balancing multi-objective goals such as sensing frequency, inference accuracy, and communication reliability. Unlike static heuristics, RL-based frameworks can dynamically adjust a device's conformity to task demands based on real-time factors like the state of charge and future energy forecasts.

Recent applications demonstrate the versatility of RL in optimizing complex IoT operations. For instance, in solar-powered UAV-assisted networks, *Yi et al.* utilize a compound-action meta-RL approach to jointly optimize flight trajectories, sensor node scheduling, and data offloading strategies [38]. Similar Meta-RL techniques enable fast adaptation for newly deployed sensors by exploiting knowledge from other locations to overcome the "cold-start" problem. In the wearable domain, the *GEM-RL* framework by *Basaklar et al.* provides generalized energy management to handle varying workloads and energy availability [3]. Furthermore, simulation studies by *Ping and Nixon* indicate that RL-optimized TinyML systems can improve battery life by up to 22.86% in image-based anomaly detection tasks compared to static optimization approaches [25].

Despite these successes, a critical assessment reveals that existing RL solutions rarely scale to the heavy compute requirements of vision workloads. Most current research focuses on scalar sensors (e.g., temperature or pressure) and low-power sensing tasks rather than high-dimensional image processing. Furthermore, many frameworks fail to jointly optimize inference and communication, often treating these as independent scheduling problems rather than a unified pipeline. While current methods enable inference, they often assume stable data arrival and scalar inputs; consequently, adaptive policies are required to handle the high variability of energy harvesting in more

demanding computational contexts.

Collectively, prior work demonstrates progress in TinyML deployment, energy-aware scheduling, and reinforcement learning-based adaptation. However, these studies remain fragmented, focusing on isolated components (sensing, inference, or communication). Few approaches consider joint optimization for vision-based edge devices under stochastic energy harvesting, motivating the framework proposed in this thesis.

Chapter 3

System Model

This chapter presents a formal mathematical model of the energy harvesting camera system considered in this thesis. The model characterizes the system operation in discrete time, capturing the dynamics of energy harvesting, battery storage, workload generation, data buffering, and energy consumption. The formulation establishes the foundation for the reinforcement learning problem introduced in Chapter 4.

3.1 Time Model

System operation is modeled in discrete time steps indexed by $t \in \{0, 1, 2, \dots\}$. Each time step corresponds to a fixed-duration interval Δt, during which sensing, processing, transmission, and energy harvesting decisions are executed.

An episode consists of $T_{\max}$ consecutive time steps and represents a complete operational cycle. This duration is chosen to capture the diurnal variability of solar energy availability, which is characteristic of outdoor energy harvesting deployments.

3.2 Energy Harvesting Model

Energy is harvested from ambient solar radiation using a photovoltaic (PV) panel characterized by surface area A_{panel} and conversion efficiency η_{solar}. The amount of energy harvested during time step t is given by

$$E_{\text{harv}}(t) = \eta_{\text{solar}} \, A_{\text{panel}} \, \text{GHI}(t) \, \Delta t, \tag{3.1}$$

where $\text{GHI}(t)$ denotes the global horizontal irradiance (W/m^2) at time t.

The irradiance process exhibits strong temporal variability driven by daily solar cycles and weather conditions. In particular, $\text{GHI}(t)$ is zero during nighttime intervals and peaks around midday. Historical irradiance traces are used to model realistic solar energy availability.

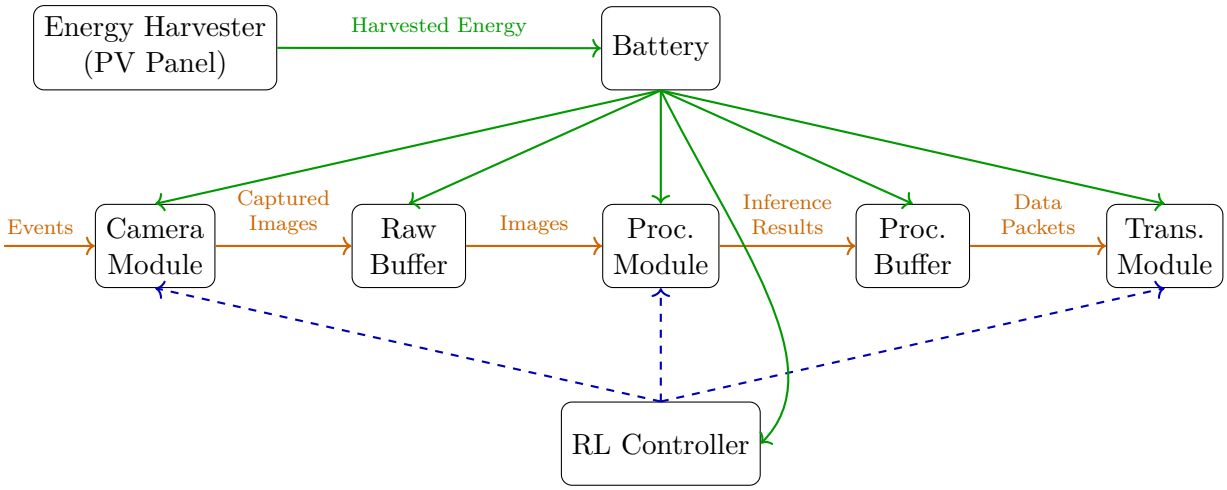

Figure 3.1: Layered system architecture of the energy harvesting camera platform. The top layer represents energy harvesting and storage, the middle layer illustrates the data processing pipeline, and the bottom layer depicts the energy-aware reinforcement learning controller.

3.3 Battery Model

The system is equipped with a rechargeable battery that serves as the sole energy storage component. Let $B(t)$ denote the battery energy level (Joules) at the beginning of time step t, and let $B_{\max}$ denote the maximum battery capacity.

The battery state evolves according to

$$B(t+1) = \min\left\{B_{\max}, \max\left\{0, B(t)(1-\lambda_{\text{leak}}) + E_{\text{harv}}(t) - E_{\text{total}}(t)\right\}\right\}, \tag{3.2}$$

where λ_{leak} denotes the self-discharge rate per time step and $E_{\text{total}}(t)$ is the total energy consumed during step t.

The battery is constrained to operate within the bounds

$$B_{\min} \leq B(t) \leq B_{\max}, \tag{3.3}$$

where $B_{\min}$ represents a critical energy threshold. Operation below $B_{\min}$ is considered unsafe and may result in system failure.

For convenience, the normalized state of charge (SoC) is defined as

$$\text{SoC}(t) = \frac{B(t)}{B_{\max}}. \tag{3.4}$$

At the start of each episode, the battery is initialized to a predefined initial state of charge $\text{SoC}(0)$.

3.4 Workload and Buffer Management

3.4.1 Dual-Buffer Architecture

The system maintains two first-in-first-out (FIFO) buffers to manage image data throughout the processing pipeline:

1. **Raw Image Buffer**: Stores captured images awaiting processing, with maximum capacity $Q_{\text{raw}}^{\max}$.
2. **Processed Image Buffer**: Stores inference results pending transmission, with maximum capacity $Q_{\text{proc}}^{\max}$.

Let $Q_{\text{raw}}(t)$ and $Q_{\text{proc}}(t)$ denote the buffer occupancies at time t. Buffer overflow occurs when

$$Q_{\text{raw}}(t) > Q_{\text{raw}}^{\max} \quad \text{or} \quad Q_{\text{proc}}(t) > Q_{\text{proc}}^{\max}. \tag{3.5}$$

When overflow occurs, excess data are discarded following a FIFO discipline, potentially resulting in loss of information.

3.4.2 Event Generation Model

The system monitors the environment for surveillance events. At each time step, an event occurs independently with probability p_{event}. Events are categorized as:

- **Important events**, occurring with probability $p_{\text{important}}$
- **Regular events**, occurring with probability $1 - p_{\text{important}}$

Each event generates a burst of image captures. The number of images generated per event follows a distribution with mean μ_{images}.

Each captured image i is associated with metadata

$$\mathcal{M}_i = (\text{id}_i, \text{event\_id}, \text{is\_important}, t_{\text{capture}}, m_{\text{proc}}), \tag{3.6}$$

where m_{proc} denotes the processing model applied to the image.

Events are subject to a delivery deadline. An important event is considered successfully handled only if at least one accurate inference result is processed and transmitted within τ_{event} time steps of event occurrence. Failure to meet this deadline results in the event being classified as missed.

3.5 Control Actions and Energy Consumption

At each time step, the controller selects a composite action

$$\mathbf{a}(t) = \Big(a_{\text{sleep}}, a_{\text{cap}}, a_{\text{proc}}, a_{\text{tx}}\Big), \tag{3.7}$$

where each component represents a discrete operational mode:

- a_{sleep}: sleep or standby power mode
- a_{cap}: image capture mode
- a_{proc}: processing model selection
- a_{tx}: transmission activation mode

The resulting action space has cardinality

$$|\mathcal{A}| = N_{\text{sleep}} \times N_{\text{cap}} \times N_{\text{proc}} \times N_{\text{tx}}. \tag{3.8}$$

Each action component incurs an energy cost. The total energy consumption during time step t is given by

$$E_{\text{total}}(t) = E_{\text{standby}}(t) + E_{\text{cap}}(t) + E_{\text{proc}}(t) + E_{\text{tx}}(t), \tag{3.9}$$

where each term corresponds to the energy consumption of the respective subsystem.

Energy consumption depends on the selected operational modes, the number of images captured, processed, or transmitted, and potential transition costs incurred when activating subsystems from inactive states.

3.6 Performance Metrics

System performance is evaluated using multiple metrics that capture reliability, efficiency, and stability.

3.6.1 Event Delivery Rate

Let $\mathcal{E}_{\text{important}}$ denote the set of important events during an episode. The event delivery rate is defined as

$$\rho_{\text{delivery}} = \frac{1}{|\mathcal{E}_{\text{important}}|} \sum_{e \in \mathcal{E}_{\text{important}}} \mathbb{I}_{\text{delivered}}(e), \tag{3.10}$$

where $\mathbb{I}_{\text{delivered}}(e)$ is an indicator function equal to 1 if event e is successfully delivered within its deadline.

3.6.2 Event Quality Score

For each delivered event e, a quality score is defined as

$$Q_{\text{event}}(e) = \min\left\{\frac{n_{\text{accurate}}(e)}{n_{\text{redundancy}}}, 1\right\}, \tag{3.11}$$

where $n_{\text{accurate}}(e)$ denotes the number of correctly classified images transmitted for event e.

The aggregate episode-level quality score is

$$Q_{\text{total}} = \sum_{e \in \mathcal{E}_{\text{delivered}}} Q_{\text{event}}(e). \tag{3.12}$$

3.6.3 Energy Efficiency

Energy efficiency is measured as the number of delivered events per unit energy:

$$\eta_{\text{energy}} = \frac{n_{\text{delivered}}}{\sum_{t=0}^{T_{\max}} E_{\text{total}}(t)}. \tag{3.13}$$

3.6.4 Episode Completion Rate

Episode completion rate is defined as the fraction of episodes that successfully complete the full simulation duration without battery depletion below the critical threshold. An episode is considered complete if the battery state remains viable throughout the entire episode:

$$\rho_{\text{completion}} = \frac{1}{N_{\text{episodes}}} \sum_{i=1}^{N_{\text{episodes}}} \mathbb{I}^{(i)}_{\text{complete}} \tag{3.14}$$

where

$$\mathbb{I}^{(i)}_{\text{complete}} = \begin{cases} 1 & \text{if } B^{(i)}(t) \geq B_{\min} \text{ for all } t \in [0, T_{\max}] \\ 0 & \text{otherwise} \end{cases} \tag{3.15}$$

and $B^{(i)}(t)$ denotes the battery state at time t during episode i. This metric captures the policy's ability to maintain long-term energy neutrality, with $\rho_{\text{completion}} = 1.0$ indicating perfect sustainability and $\rho_{\text{completion}} < 1.0$ revealing episodes where catastrophic battery depletion occurred despite the agent's control efforts.

Chapter 4

Reinforcement Learning Formulation

This chapter formulates the control of the energy harvesting camera system as a reinforcement learning (RL) problem. Due to the stochastic nature of energy availability and event generation, as well as the coupling between energy storage and workload execution, conventional rule-based control strategies are insufficient. Reinforcement learning provides a principled framework for learning adaptive control policies directly from interaction with the environment.

The system is modeled as a Markov Decision Process (MDP), where the RL agent observes the system state, selects control actions, and receives feedback in the form of a reward signal that reflects system performance objectives.

4.1 Motivation for Reinforcement Learning

Energy harvesting systems operate under significant uncertainty arising from time-varying environmental conditions and unpredictable workloads. In the considered camera platform, solar energy availability fluctuates over time, while surveillance events occur randomly and may vary in importance.

Traditional energy management approaches often rely on static thresholds or heuristics, which are difficult to tune and fail to generalize across operating conditions. In contrast, reinforcement learning enables the controller to:

- Adapt its decisions to changing energy and workload conditions,
- Learn long-term trade-offs between energy consumption and event delivery,
- Exploit temporal correlations in solar energy availability, and
- Optimize system performance without explicit modeling of future dynamics.

These properties make reinforcement learning particularly suitable for autonomous energy harvesting platforms.

4.2 Markov Decision Process Definition

The system is formulated as a discrete-time Markov Decision Process defined by the tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma), \tag{4.1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}$ represents the state transition probabilities, $\mathcal{R}$ is the reward function, and $\gamma \in (0, 1]$ is the discount factor.

4.2.1 State Space

At each time step t, the RL agent observes a system state $s(t) \in \mathcal{S}$ that summarizes all relevant information required for decision making. The state vector is defined as

$$s(t) = \Big[\mathrm{SoC}(t),\ Q_{\mathrm{raw}}(t),\ Q_{\mathrm{proc}}(t),\ \hat{E}_{\mathrm{harv}}(t),\ \hat{L}_{\mathrm{event}}(t)\Big], \tag{4.2}$$

where:

- $\mathrm{SoC}(t)$ is the normalized battery state of charge,
- $Q_{\mathrm{raw}}(t)$ and $Q_{\mathrm{proc}}(t)$ denote the occupancies of the raw and processed data buffers,
- $\hat{E}_{\mathrm{harv}}(t)$ is an estimate of harvested energy availability, derived from recent irradiance measurements,
- $\hat{L}_{\mathrm{event}}(t)$ represents a workload indicator reflecting recent event activity.

This state representation balances expressiveness and tractability while preserving the Markov property.

4.2.2 Action Space

At each time step, the agent selects a control action $a(t) \in \mathcal{A}$ that governs the operation of system modules. The action is defined as a composite vector:

$$a(t) = \Big(a_{\mathrm{cap}}(t), a_{\mathrm{proc}}(t), a_{\mathrm{tx}}(t)\Big), \tag{4.3}$$

where:

- $a_{\mathrm{cap}}(t)$ determines the image capture mode,

- $a_{\text{proc}}(t)$ selects the processing model or processing intensity,
- $a_{\text{tx}}(t)$ controls data transmission activity.

Each action component takes values from a finite discrete set, resulting in a finite but combinatorial action space. Actions are constrained by the current battery level to prevent unsafe operation.

4.2.3 State Transition Dynamics

The system dynamics are governed by the interaction between energy harvesting, battery evolution, workload processing, and buffering. Given the current state $s(t)$ and action $a(t)$, the next state $s(t+1)$ evolves according to the battery dynamics in (3.2), buffer update rules, and stochastic energy and event processes.

The transition probabilities $\mathcal{P}(s(t+1) \mid s(t), a(t))$ are unknown and time-varying, which motivates the use of model-free reinforcement learning.

4.3 Reward Function Design

The reward function plays a critical role in guiding the learning process toward desirable system behavior. The design objective is to encourage reliable event delivery while maintaining long-term energy sustainability and avoiding buffer overflow or battery depletion.

The instantaneous reward at time step t is defined as

$$r(t) = r_{\text{event}}(t) - r_{\text{energy}}(t) - r_{\text{buffer}}(t) - r_{\text{failure}}(t). \tag{4.4}$$

Each component is described below.

4.3.1 Event Reward

A positive reward is granted when an important event is successfully processed and transmitted within its deadline:

$$r_{\text{event}}(t) = \begin{cases} R_{\text{success}}, & \text{if an important event is delivered,} \\ 0, & \text{otherwise.} \end{cases} \tag{4.5}$$

This term directly incentivizes timely and accurate event handling.

4.3.2 Energy Consumption Penalty

To discourage excessive energy usage, a penalty proportional to the consumed energy is applied:

$$r_{\text{energy}}(t) = \alpha_E \, E_{\text{total}}(t), \tag{4.6}$$

where α_E is a scaling factor that controls the trade-off between performance and energy efficiency.

4.3.3 Buffer Penalty

Buffer congestion and overflow are penalized to promote stable system operation:

$$r_{\text{buffer}}(t) = \alpha_Q \left(\frac{Q_{\text{raw}}(t)}{Q_{\text{raw}}^{\max}} + \frac{Q_{\text{proc}}(t)}{Q_{\text{proc}}^{\max}} \right). \tag{4.7}$$

This term encourages the agent to regulate workload admission and processing rates.

4.3.4 Failure Penalty

Critical system failures, such as battery depletion below $B_{\min}$, incur a large negative penalty:

$$r_{\text{failure}}(t) = \begin{cases} R_{\text{fail}}, & \text{if } B(t) < B_{\min}, \\ 0, & \text{otherwise.} \end{cases} \tag{4.8}$$

This penalty ensures that energy sustainability is treated as a hard constraint.

4.4 Learning Objective

The goal of the RL agent is to learn a policy $\pi(a \mid s)$ that maximizes the expected discounted return:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T_{\max}} \gamma^t r(t) \right], \tag{4.9}$$

where γ is the discount factor. A value of γ close to one emphasizes long-term energy sustainability and future event handling.

4.5 Training Procedure

The agent interacts with a simulated environment constructed from the system model described in Chapter 3. Training is conducted over multiple episodes, each corresponding to a full operational cycle.

During training, exploration is encouraged through stochastic action selection, while unsafe actions that would violate battery constraints are masked. Model parameters are updated using sampled transitions $(s(t), a(t), r(t), s(t+1))$.

The proposed RL formulation captures the essential trade-offs of energy harvesting systems: balancing immediate workload demands against future energy availability. By explicitly incorporating battery state, buffer occupancy, and workload indicators into the state representation, the agent learns energy-aware policies that adapt to environmental dynamics.

This formulation is general and can be extended to other energy harvesting sensing platforms by modifying the state and action definitions while preserving the overall learning framework.

Chapter 5

Implementation and Training

This chapter describes the practical implementation of the reinforcement learning system, including the simulation environment design, training methodology, hyperparameter configuration, and curriculum learning strategy. The implementation translates the theoretical formulation from Chapter 4 into a functional training pipeline capable of learning effective energy management policies.

5.1 Simulation Environment

5.1.1 Environment Architecture

The core of the training pipeline is a high-fidelity simulation environment implemented within the Gymnasium framework, providing a standardized Markov Decision Process (MDP) interface for reinforcement learning. This environment serves as a digital twin of the energy harvesting camera system, encapsulating the complex physical and stochastic dynamics described in Chapter 3. Specifically, the architecture integrates battery evolution models—accounting for charge-discharge efficiency and self-discharge leakage—with a dual-buffer management system that handles First-In-First-Out (FIFO) data flow and overflow conditions. To represent real-world operation, the environment manages stochastic event generation, image capture, and variable energy consumption based on the selected operational modes. Following the Gymnasium API standards, the environment is driven by two primary methods: `reset()`, which initializes the system state with randomized starting conditions to ensure diverse training scenarios, and `step()`, which executes an agent's composite action to update the internal state, compute the transition reward, and return the subsequent observation.

5.1.2 Solar Irradiance Data

Realistic solar energy harvesting is achieved by grounding the simulation in high-resolution historical data from the National Solar Radiation Database (NSRDB), maintained by the National Renewable Energy Laboratory (NREL). Specifically, the environment utilizes the NSRDB "USA & Americas" dataset for the year 2023, targeting the geographic coordinates associated with Location ID 3762824. These data capture the authentic diurnal and seasonal variability required to evaluate the agent's long-term energy sustainability under realistic environmental stress.

The data preprocessing pipeline ingest the raw Global Horizontal Irradiance (GHI) measurements, which are provided by the NSRDB at a 10-minute temporal resolution. To align the environmental input with the system model's operational granularity, the pipeline applies linear interpolation to map these 10-minute intervals to a 1-minute resolution, matching the discrete time step Δt. All values are normalized to a standard $[0, 1000]$ W/m^2 range to maintain consistency during neural network training. To ensure the policy generalizes across diverse weather patterns rather than over-fitting to a single sequence, the environment implements wraparound indexing and randomly selects starting time indices for each episode. This approach ensures that the agent is exposed to a comprehensive spectrum of solar conditions, ranging from clear-sky solar peaks to the high-uncertainty periods characteristic of cloud-induced irradiance fluctuations.

5.1.3 Inference Subsystem: MobileNetV2 Architecture

The processing module within the simulation environment is implemented as a deep convolutional neural network (CNN) optimized for mobile and edge deployment. This subsystem provides the inference capabilities required to classify wildlife species within captured images. Given the energy constraints of an autonomous platform, MobileNetV2 was selected as the backbone architecture due to its use of inverted residual blocks and depthwise separable convolutions, which significantly reduce the computational FLOP count while maintaining high accuracy. The classification head was customized for the Serengeti workload with a Global Average Pooling layer followed by two dense layers of 512 and 256 units, respectively, utilizing ReLU activations and L2 regularization ($\lambda = 0.001$). The total trainable parameter count for the custom head and fine-tuned layers is approximately 793,876. The performance characteristics of this model directly inform the energy costs $E_{\text{proc}}(t)$ and reward signals $r_{\text{event}}(t)$ used in the environment; specifically, the RL agent's choice of processing intensity corresponds to the execution of this model.

5.1.4 Dataset and Training Strategy

The inference model was trained on a curated subset of the Snapshot Serengeti dataset [35], targeting 20 mammalian species common to the African savanna. The training corpus consists of 24,673 high-resolution images. To ensure robust performance despite the inherent class imbalance of wildlife distributions, we employed a stratified sampling strategy, partitioning the data into a training set (70%), validation set (15%), and test set (15%). Class imbalance was further addressed by computing balanced class weights to ensure rare species contribute significantly to the loss:

$$w_j = \frac{N}{C \times n_j} \tag{5.1}$$

where N is the total samples, C is the number of classes (20), and n_j is the number of samples in class j. The model utilized a two-phase training protocol: an initial transfer learning phase where base weights were frozen for 20 epochs, followed by a fine-tuning phase where layers from index 100 onwards were unfrozen. This resulted in a Top-1 Test Accuracy of approximately 94%, which the simulation uses to determine the probability of successful event delivery.

Table 5.1: Inference Model Hyperparameters

Hyperparameter	**Value**
Input Resolution	$224 \times 224 \times 3$
Backbone	MobileNetV2 (ImageNet)
Optimizer	Adam
Base Learning Rate	10^{-3} (Phase 1), 10^{-4} (Phase 2)
Batch Size	32
Dropout Rate	0.3
L2 Regularization (λ)	0.001
Target Classes	20

5.1.5 Subsystem Energy Modeling

The fidelity of the simulation environment relies on its ability to accurately translate the agent's high-level control decisions into physical energy consumption. To achieve this, the environment implements a suite of state-aware energy models for the capturing, processing, and transmission subsystems. These models are designed to account for the non-negligible overhead of toggling hardware components between idle and active states, thereby penalizing frequent mode switching and encouraging the agent to learn more sustainable operational sequences. The total energy consumed during any given time step, $E_{\text{total}}(t)$, is the summation of these three specific components.

Image Capture Energy Model The energy required for image acquisition, E_t^{cap}, is a function of the selected capture mode a_t and the system's previous state. As formulated in Equation (5.2), the environment distinguishes between three distinct modes: inactive ($a_t = 0$), low-power sensing ($a_t = 1$), and high-speed burst capture ($a_t = 2$). A critical feature of this model is the inclusion of a software initialization penalty, E_{init}, which is triggered by the indicator function $\mathbb{I}(a_{t-1} = 0)$ whenever the camera module transitions from a standby state. For active modes, the model scales energy consumption based on the number of images captured, N_t, while accounting for mode-specific active costs, logging overheads, and the standby energy required between frames in high-speed mode.

$$E_t^{\text{cap}} = \begin{cases} 0 & a_t = 0 \\ \mathbb{I}(a_{t-1} = 0)E_{\text{init}} + N_t\, e_{\text{active}}^{L} + E_{\log}^{L} & a_t = 1 \\ \mathbb{I}(a_{t-1} = 0)E_{\text{init}} + E_{\text{start}}^{H} + N_t\, e_{\text{cap}}^{H} + (N_t - 1)e_{\text{stby}}^{H} + E_{\log}^{H} & a_t = 2 \end{cases} \tag{5.2}$$

Image Processing Energy Model The energy consumed during inference, E_t^{proc}, is determined by the number of images residing in the buffer to be processed, N_t^{proc}, and the complexity of the selected MobileNetV2 configuration. Beyond the per-image inference cost e_{proc} and fixed logging overheads $E_{\log}$, the environment explicitly models the energy required to load specific model weights into the processor's memory. As shown in Equation (5.3), indicator functions are used to apply initialization penalties $E_{\text{init}}^{\text{S}}$ or $E_{\text{init}}^{\text{C}}$ when the subsystem transitions from an idle state to a simple or complex processing mode, respectively. This ensures the RL agent understands the trade-offs between maintaining a processing state and intermittently activating the inference engine.

$$\begin{aligned} E_t^{\text{proc}} = {} & N_t^{\text{proc}}\, e_{\text{proc}}(a_t^{\text{proc}}) + E_{\log}(a_t^{\text{proc}}) \\ & + \mathbb{I}(a_{t-1}^{\text{proc}} = 0 \wedge a_t^{\text{proc}} = 1)\, E_{\text{init}}^{\text{S}} \\ & + \mathbb{I}(a_{t-1}^{\text{proc}} = 0 \wedge a_t^{\text{proc}} = 2)\, E_{\text{init}}^{\text{C}} \end{aligned} \tag{5.3}$$

Data Transmission Energy Model The energy profile of the wireless communication subsystem, E_t^{tx}, captures the costs associated with operating a high-power transmission dongle. The model presented in Equation (5.4) accounts for the energy required to establish a network connection ($E_{\text{init}}^{\text{tx}}$) and the baseline energy needed to maintain the active link ($E_{\text{dongle},t}$). Furthermore, the model penalizes deactivation by applying a shutdown energy $E_{\text{off}}^{\text{tx}}$ if the subsystem was active in the previous step but is set to idle in the current step. When active, the energy scales with the number of packets transmitted, N_t^{tx}, multiplied by the per-packet cost e_{tx}, allowing the agent to

learn the optimal batch sizes for efficient data offloading.

$$E_t^{\text{tx}} = \begin{cases} \mathbb{I}(a_{t-1}^{\text{tx}} > 0) E_{\text{off}}^{\text{tx}} + 0 & a_t^{\text{tx}} = 0 \\ \mathbb{I}(a_{t-1}^{\text{tx}} = 0) E_{\text{init}}^{\text{tx}} + E_{\text{dongle},t} + N_t^{\text{tx}} \, e_{\text{tx}}(a_t^{\text{tx}}) & a_t^{\text{tx}} > 0 \end{cases} \tag{5.4}$$

where the WiFi Dongle standby energy $E_{\text{dongle},t}$ is defined as:

$$E_{\text{dongle},t} = \begin{cases} (\Delta t - T_{\text{startup}}) P_{\text{dongle}} & \text{if } a_{t-1}^{\text{tx}} = 0 \\ \Delta t \cdot P_{\text{dongle}} & \text{if } a_{t-1}^{\text{tx}} > 0 \end{cases} \tag{5.5}$$

5.1.6 State Observation Construction

At each discrete time step, the environment provides the agent with a 44-dimensional observation vector that characterizes the current internal and external conditions. This vector is constructed through a multi-stage process, beginning with the extraction and normalization of core state variables, including the battery SoC, buffer occupancies, and specific processed image counts. To capture the system's previous decisions, the action tuple is converted into a concatenated series of one-hot vectors. Furthermore, to provide the agent with a sense of temporal trends, the environment computes exponential moving averages for the battery level and buffer congestion using fixed-window deque structures. For example, the battery trend is derived from a 10-step window, $\text{MA}_{\text{battery}} = \frac{1}{10} \sum_{i=0}^{9} \text{SoC}(t-i)$, allowing the policy to distinguish between steady energy depletion and active charging. Finally, the state incorporates a safety flag reflecting hardware interventions and a sliding window of the 24 most recent GHI measurements, all of which are concatenated into a `float32` NumPy array for optimized processing during the neural network's forward pass.

5.1.7 Reward Computation Pipeline

The reward function is computed through a modular pipeline designed to guide the agent toward reliable surveillance and energy-neutral operation. The process begins with event finalization, where any event exceeding its timeout deadline, τ_{event}, is assessed for successful delivery. A significant positive reward of $+6.0$ is granted for important events that resulted in at least one accurate inference, while a failure to meet the deadline incurs a -3.0 miss penalty. For successful deliveries, a quality bonus is awarded based on the number of accurate results, $r_{\text{quality}} = 0.05 \times \min(n_{\text{accurate}}, n_{\text{redundancy}})$. To promote stability, the pipeline accumulates penalties for buffer congestion and unsafe battery levels. Additionally, small shaping rewards are provided for processing and transmission activity to alleviate the challenges of sparse rewards. The total instantaneous reward is finally clipped to the range of $[-10, +10]$, ensuring gradient stability during the policy update phase.

5.1.8 Safety Constraints

To mitigate the risk of catastrophic system failure during the agent's initial exploration phase, the environment incorporates a "Soft Safety Shield" that acts as a heuristic safeguard. This mechanism monitors the battery SoC and intervenes with the agent's proposed action if energy levels become critical. As detailed in Algorithm 1, the shield implements two tiers of intervention: a scaling tier that reduces action intensity when the SoC drops below 12%, and an emergency tier that forces a minimal-energy standby mode if the SoC falls below 5%. Crucially, the occurrence of an intervention is communicated back to the agent via the safety flag in the observation vector. This allows the policy to learn the boundaries of safe operation, eventually discovering a strategy that avoids triggering these constraints altogether.

Algorithm 1 Soft Safety Shield Mechanism

Input: Battery SoC, proposed action $\mathbf{a}$
Output: Safe action $\mathbf{a}'$, intervention flag
if SoC < 0.12 **and** SoC ≥ 0.05 **then**
    Scale down action intensity by factor 0.5
    Return scaled action, `soft_intervention`
else if SoC < 0.05 **then**
    Force minimal energy action: $(a_{\text{sleep}} = 2, a_{\text{cap}} = 0, a_{\text{proc}} = 0, a_{\text{tx}} = 0)$
    Return emergency action, `hard_intervention`
else
    Return original action, `no_intervention`
end if

5.2 PPO Implementation Details

5.2.1 Neural Network Architecture

The Proximal Policy Optimization (PPO)[33] framework is implemented using a decoupled Actor-Critic architecture to ensure that policy improvement and value estimation do not interfere with one another during the gradient update process. Both networks process the same 44-dimensional state observation vector but are optimized toward distinct objectives: the actor learns to map states to optimal control actions, while the critic learns to estimate the expected cumulative reward, or value, of being in a specific state. This separation of concerns is fundamental for maintaining stability in environments with non-stationary reward distributions and complex action dynamics.

Actor Network Architecture

The actor network is designed as a hierarchical structure that exploits the factored nature of the action space. Rather than employing a flat categorical distribution over all

54 possible joint actions—which would lead to a sparse and difficult-to-optimize probability space—the network utilizes a shared feature extraction backbone that branches into four independent output heads. As specified in the architectural parameters, the backbone consists of two hidden layers with 256 neurons each, utilizing Rectified Linear Unit (ReLU) activations to extract task-agnostic features.

These shared features are then processed by four specialized linear layers that produce logits for sleep mode, image capture, processing intensity, and data transmission, these logits are converted into independent categorical distributions via the Softmax function. By assuming independence between action components during the sampling phase, the policy effectively reduces the parameter count and improves expressiveness. This factorization also facilitates efficient computation of log probabilities and policy entropy, the latter of which is crucial for regulating exploration through the curriculum stages. In total, the actor network comprises 80,139 trainable parameters, striking a balance between model capacity and computational efficiency for edge-integrated training.

Critic Network Architecture

The critic network serves as the value baseline for the PPO algorithm, estimating the state-value function $V_\phi(\mathbf{s})$. Architecturally, the critic mirrors the actor's backbone with two hidden layers of 256 units, but it remains entirely independent in terms of weights to prevent gradient conflicts between the policy and value updates. The final output is a single scalar value produced by a linear head without an activation function, allowing the network to predict unbounded values corresponding to the expected discounted returns. This network configuration results in 77,569 parameters, bringing the total combined agent parameter count to 157,708.

Weight Initialization and Forward Pass

Proper weight initialization is critical for the stability of the PPO agent, particularly in the early stages of learning where random actions must not lead to immediate policy collapse. We employ orthogonal initialization, which preserves the norm of gradients during backpropagation. To account for the characteristics of the ReLU activation function, a gain of $\sqrt{2}$ is applied to the hidden layers. Conversely, a significantly smaller gain of 0.01 is applied to the actor's output heads. This ensures that the initial policy is nearly uniform, preventing the agent from prematurely converging on a suboptimal deterministic strategy.

During the execution phase, the agent operates in two distinct modes. For training rollouts, actions are sampled stochastically from the categorical distributions to encourage exploration. For evaluation and deployment, the agent switches to a de-

terministic mode, where the action with the highest probability (the argmax of the logits) is selected. This dual-mode forward pass, as described in the accompanying algorithms, allows for robust data collection while ensuring peak performance during testing.

5.2.2 Rollout Buffer and Advantage Estimation

The system utilizes a rollout buffer to store on-policy experience, which is entirely cleared after each policy update. This buffer maintains arrays for states, actions, log probabilities, and value estimates for $N = 2048$ transitions. Unlike off-policy methods that rely on replay buffers, the on-policy nature of PPO requires fresh data that accurately reflects the current policy's behavior.

To optimize the policy gradient, we implement Generalized Advantage Estimation (GAE). GAE provides a controllable bias-variance trade-off through the λ parameter. The process involves computing temporal difference (TD) residuals and then applying a backward recursion to estimate the advantage of each action, representing how much better an action performed compared to the critic's baseline expectation. Crucially, we distinguish between natural episode termination and truncation due to time limits; for the latter, we bootstrap the remaining value using the critic's prediction to ensure that the agent is not penalized for reaching the simulation's maximum duration.

5.2.3 Reward and Advantage Normalization

To maintain gradient stability across the diverse curriculum stages, the implementation utilizes two layers of normalization. First, the raw rewards received from the environment are normalized using running mean and standard deviation statistics. This prevents the value function from being overwhelmed by shifting reward scales as the agent improves. Second, once the advantages are computed within the rollout buffer, they are normalized to zero mean and unit variance. This second layer of normalization ensures that the PPO clipping mechanism remains effective regardless of the advantage magnitude, providing a consistent "trust region" for policy updates.

5.2.4 Optimization Process

The optimization phase is triggered whenever the rollout buffer reaches capacity. The agent performs five epochs of stochastic gradient descent using the Adam optimizer on minibatches of size 64. The objective function is a composite of three terms: the clipped surrogate policy loss, the value function loss, and an entropy bonus.

The clipped surrogate loss is the core innovation of PPO, as it prevents the new policy from deviating too far from the old policy by clipping the probability ratio within

a range of $[1-\epsilon, 1+\epsilon]$. Simultaneously, the value function loss is also clipped to prevent catastrophic updates to the critic. The entropy bonus is weighted via a coefficient c_S, which is curriculum-adaptive—starting high to encourage broad exploration in early stages and decreasing as the agent approaches expert performance. This comprehensive optimization loop, combined with global gradient clipping to a maximum norm of 0.5, ensures that the learning process remains stable and monotonically improves over millions of training steps.

Table 5.2: Implementation Hyperparameters

Component	**Parameter**	**Value**
Network	Hidden dimensions	256
	Hidden layers	2
	Activation	ReLU
PPO	Learning rate α	3×10^{-4}
	Discount γ	0.99
	GAE λ	0.95
	Clip ϵ	0.2
	Value clip ϵ_V	0.2
	Max gradient norm	0.5
Training	Rollout buffer size N	2048
	Minibatch size B	64
	Update epochs K	5
	Optimizer	Adam
Loss Weights	Value coef. c_V	0.5
	Entropy coef. c_S	0.1 (curriculum-adaptive)
Normalization	Advantage normalization	Yes
	Reward normalization	Running statistics

5.3 Curriculum Learning Strategy

Training a reinforcement learning policy from a state of random initialization within the full complexity of the Serengeti environment often results in poor convergence. This difficulty arises from the combination of sparse rewards—where successful event deliveries are rare—and complex credit assignment, where the agent must correlate an action taken many steps prior with a eventual battery depletion or buffer overflow. To overcome these challenges, we employ a curriculum learning strategy that progressively increases the task difficulty, allowing the agent to build a foundational understanding of energy management before facing the high-workload demands of the expert environment.

5.3.1 Curriculum Design Principles

The curriculum is structured around a transition from a highly permissive "tutorial" environment to a resource-constrained "expert" deployment. This progression is governed by six key design principles. First, the event frequency, p_{event}, is kept low in the early stages, allowing the agent to learn the relationship between solar availability and battery charging without the pressure of a continuous workload. Second, we utilize smaller average event sizes, μ_{images}, to minimize initial buffer congestion. To prevent premature episode termination, early stages also utilize a higher initial state of charge and relaxed delivery constraints, such as longer timeouts, τ_{event}, and expanded transmission capacities.

As the agent demonstrates proficiency, we gradually introduce more stringent performance requirements. This includes increasing the miss penalties, r_{missed}, to enforce higher delivery reliability and reducing the entropy coefficient, c_S. This process of entropy annealing is critical; by starting with a high coefficient (0.30), we encourage the broad exploration of the action space, while the final reduction to 0.10 in the expert stage forces the policy to transition from exploration to the exploitation of learned optimal behaviors.

5.3.2 Stage Progression and Parameters

The curriculum is implemented across six distinct stages, ranging from "Tutorial Ultra Easy" to "Expert," as summarized in Table 5.3. The total training duration of 4,700 episodes is distributed such that more time is allocated to the harder stages, where the policy must fine-tune its response to high-frequency bursts and critical energy thresholds. By the final stage, the agent is required to manage an event probability of 0.30 and event sizes of up to 100 images, representing the full operational stress of the Serengeti camera trap deployment.

Table 5.3: Curriculum Learning Stages and Environmental Parameters

Stage	**Episodes**	**SoC**	p_{event}	μ_{images}	$n_{\text{tx}}^{\max}$	τ_{event}	r_{missed}	c_S
Tutorial Ultra Easy	200	0.99	0.05	10	10	500	-1.0	0.30
Tutorial	500	0.95	0.10	20	9	300	-2.0	0.20
Easy	800	0.90	0.15	30	9	300	-2.5	0.18
Medium	1000	0.80	0.20	50	7	250	-3.0	0.15
Hard	1000	0.60	0.25	75	6	200	-3.5	0.12
Expert	1200	0.50	0.30	100	5	150	-4.0	0.10
Total	**4700**							

5.3.3 Stage Transition Protocol

Transitions between curriculum stages occur automatically upon the completion of the prescribed number of episodes, utilizing a "warm-start" methodology to preserve learned knowledge. At each transition point, a comprehensive system checkpoint is saved, including both the policy and critic network weights. The environment is then reconfigured with the updated system parameters, and the Adam optimizer moments are optionally reset to prevent momentum mismatch caused by the sudden shift in reward distributions. Crucially, the network parameters are retained rather than reinitialized, allowing the agent to leverage previously acquired skills—such as basic power-saving modes—while adapting to the increased difficulty of the new stage. This iterative refinement ensures a stable training trajectory and facilitates the emergence of sophisticated energy-aware policies that would otherwise be unattainable through direct training on the expert task.

5.4 Training Procedure

5.4.1 Episode Execution

The training process is structured around the iterative execution of operational episodes, each representing a complete diurnal cycle. As described in Algorithm 2, each episode begins with an environment reset that initializes the system state s_0. During each time step, the agent samples a composite action $\mathbf{a}$ from its current policy π_θ and records the associated log probability and state value estimate. These transitions—comprising the state, action, reward, and termination status—are stored in the rollout buffer. When the buffer reaches its capacity of 2,048 transitions, the system computes the Generalized Advantage Estimation (GAE) and performs a PPO update before resetting the buffer for the next rollout. This cycle continues until the episode terminates through either natural completion (reaching $T_{\max}$) or catastrophic battery depletion.

Algorithm 2 Training Episode Sequence

Reset environment: $s_0, \text{info} \leftarrow$ `env.reset()`
Initialize episode accumulators: $R_{\text{episode}} = 0$, step $= 0$
while not done **and** step $< T_{\max}$ **do**
  Sample action: $\mathbf{a} \sim \pi_\theta(\cdot|s)$
  Compute log probability: $\log p = \log \pi_\theta(\mathbf{a}|s)$
  Compute state value: $v = V_\phi(s)$
  Execute action: $s', r, \text{terminated}, \text{truncated}, \text{info} \leftarrow$ `env.step`$(\mathbf{a})$
  Store transition: $(s, \mathbf{a}, \log p, r, v, \text{done})$ in buffer
  Update accumulators: $R_{\text{episode}} \leftarrow R_{\text{episode}} + r$
  Update state: $s \leftarrow s'$, step $\leftarrow$ step $+1$
  done $\leftarrow$ terminated $\vee$ truncated
  if buffer is full **then**
    Compute $V_{\text{last}} = V_\phi(s')$ if not done, else 0
    Finish episode in buffer (compute GAE)
    Perform PPO update
    Reset buffer
  end if
end while
Return: R_{episode}, episode statistics

5.4.2 Evaluation Protocol

To assess the actual performance of the learned policy without the noise of stochastic exploration, we implement a periodic evaluation protocol. Every 100 episodes during the training process, the agent's exploration is frozen, and the policy is switched to a deterministic mode using greedy action selection—specifically, selecting the argmax over the policy distributions. The agent is then required to execute $n_{\text{eval}} = 10$ full episodes using a set of fixed random seeds that differ from those used in training. During these runs, we compute a suite of performance metrics including the mean episode reward, the event delivery rate for important events, overall energy efficiency, and the final battery state of charge (SoC) distribution. These results are logged for diagnostic comparison to ensure the policy is generalizing effectively to unseen environmental conditions.

5.4.3 Convergence Monitoring

Monitoring the health and convergence of the training process is essential for identifying policy instability or degenerate learning patterns. We utilize primary metrics, such as the 100-episode moving average of rewards and the event delivery rate, to confirm that the agent is progressing toward the 70% success threshold required for the expert stage. Additionally, the "explained variance" of the critic's value function is tracked; a value consistently above 0.5 indicates that the critic is successfully capturing the underlying

reward structure of the environment.

Secondary indicators provide further insight into the optimization dynamics. We monitor policy and value loss for stabilization, ensuring that the policy entropy gradually decreases as the agent becomes more confident in its decisions. Furthermore, the KL divergence is monitored to ensure it remains below 0.02, indicating stable, incremental updates that do not violate the PPO trust region. These indicators allow for the early detection of failure modes, such as reward collapse—where the agent learns a "do-nothing" strategy to avoid penalties—or value overestimation, where the critic predicts unrealistically high returns that can lead to exploding gradients.

5.5 Diagnostic and Debugging Tools

5.5.1 Pre-Flight Checks

To ensure the integrity of the training pipeline before initiating large-scale optimization, the system performs a series of automated pre-flight checks. These diagnostic tests verify that the environment is internally consistent and that the parameters of the curriculum stages represent achievable operational goals. Specifically, the checks validate the consistency of stage-wise parameters to ensure that event delivery rates are mathematically feasible given the current energy harvesting constraints. Furthermore, the system confirms that the reward surface is properly shaped, verifying that even rudimentary aggressive policies are capable of yielding positive total rewards during the initial tutorial stages. Hardware-specific constraints are also validated, ensuring that the data transmission capacity is sufficient to prevent permanent buffer saturation and that the energy harvesting model can sustain minimal standby operations without immediate battery depletion.

5.5.2 Reward Decomposition Tracking

To facilitate precise debugging of the agent's behavior, the environment implements a reward decomposition tracking system that logs the contribution of each individual reward component. This modular breakdown provides insight into how the agent is balancing its various objectives. In a successful training trajectory, the delivery reward should eventually account for over 60% of the total positive rewards, with the quality bonus serving as a secondary contributor of approximately 10–20%. The shaping rewards, which provide dense feedback for processing and transmission, are expected to decrease in relative importance as the policy learns to prioritize final event delivery. If the decomposition analysis reveals that penalties for buffer overflow or battery depletion dominate the total reward signal, it serves as a clear indicator that the agent is failing to manage its long-term energy or workload stability.

5.6 Implementation Framework

5.6.1 Software Dependencies

The implementation of the autonomous energy harvesting system is built upon a high-performance Python-based software stack, chosen for its robustness in both numerical simulation and deep learning research. As detailed in Table 5.4, the core of the system is developed using PyTorch 2.0+, which provides the necessary primitives for neural network construction and automatic differentiation. The reinforcement learning interface is managed through Gymnasium 0.29+, ensuring a standardized Markov Decision Process (MDP) structure for agent-environment interaction. Data processing and solar irradiance modeling utilize NumPy and Pandas for efficient array manipulations and time-series analysis. Finally, visualization and statistical reporting are conducted using Matplotlib and Seaborn, providing the clarity required for diagnosing complex learning behaviors.

Table 5.4: Software Dependencies and Framework Components

Library	**Purpose**
PyTorch 2.0+	Neural network implementation and automatic differentiation
Gymnasium 0.29+	RL environment interface
NumPy 1.24+	Numerical computations and array operations
Pandas 2.0+	Solar irradiance data loading and processing
Matplotlib 3.7+	Training visualization and plotting
Seaborn 0.12+	Enhanced statistical visualizations

5.6.2 Code Organization

To ensure maintainability and a clear separation of concerns, the codebase is organized into several modular components. The Core Module manages system parameters and environmental data loading, while the Energy Module encapsulates the physical models for battery dynamics and buffer management. The Environment Module provides the Gymnasium-compliant transition and reward logic, and the PPO Module contains the specific actor-critic network architectures and rollout buffer implementations. Higher-level logic is handled by the Training Module, which governs curriculum transitions, and the Diagnostics Module, which provides real-time analysis of action distributions and rewards. This modular design facilitates the independent testing of components, allows for the easy modification of specific hardware subsystems without disrupting the RL logic, and ensures that the framework can be extended for future sensing platforms.

5.6.3 Reproducibility

Reproducibility is a critical requirement for validating the performance of reinforcement learning agents. To ensure that the results reported in this thesis are consistent and verifiable, we have implemented a rigorous experimental protocol. This includes setting fixed random seeds across the NumPy, PyTorch, and Gymnasium libraries and enabling deterministic algorithms in the PyTorch backend where possible. Additionally, all solar GHI data and system parameters are version-controlled, and every training checkpoint is saved alongside its specific hyperparameter configuration. The environment state is also configured to include the episode start index, allowing for the exact replay of irradiance scenarios. Consequently, all experimental results reported in Chapter 7 are derived from fixed random seeds and identical configurations, ensuring that the observed improvements are a direct result of the proposed control strategies.

5.7 Baseline Methods and Comparative Approaches

To rigorously evaluate the performance of the proposed reinforcement learning framework, we implement and compare against multiple baseline approaches spanning three distinct categories: rule-based heuristic policies, model-based optimization methods, and alternative reinforcement learning algorithms. This section provides detailed descriptions of the architecture, decision mechanisms, and design rationale for each baseline method.

5.7.1 Heuristic Baseline Policies

Heuristic policies represent engineered control strategies that encode domain knowledge through explicit rules and thresholds. While lacking the adaptability of learned policies, well-designed heuristics often serve as strong baselines and provide interpretable decision-making processes.

Random Policy

The random policy serves as a lower-bound baseline, selecting actions uniformly at random from the discrete action space at each time step. Formally, at time t, the policy samples:

$$a_{\text{sleep}}(t) \sim \mathcal{U}\{0,1,2\}, \quad a_{\text{cap}}(t) \sim \mathcal{U}\{0,1,2\}, \quad a_{\text{proc}}(t) \sim \mathcal{U}\{0,1,2\}, \quad a_{\text{tx}}(t) \sim \mathcal{U}\{0,1\} \tag{5.6}$$

where $\mathcal{U}\{\cdot\}$ denotes the discrete uniform distribution. This baseline establishes the minimal expected performance achievable without any control strategy, serving as a sanity check that informed policies should substantially exceed.

Always Sleep Policy

The always sleep policy represents an extreme conservative strategy that prioritizes energy preservation over task execution. This policy consistently selects deep sleep mode while disabling all other subsystems:

$$\mathbf{a}(t) = (2, 0, 0, 0) \quad \forall t \tag{5.7}$$

corresponding to deep sleep with no capture, processing, or transmission. While this policy guarantees long-term energy sustainability, it achieves zero event delivery and serves primarily as a verification that the system can maintain operation indefinitely without workload execution.

Maximum Throughput Policy

In contrast to the always sleep policy, the maximum throughput policy operates at full capacity regardless of energy constraints. This energy-blind approach continuously executes:

$$\mathbf{a}(t) = (0, 2, 2, 1) \quad \forall t \tag{5.8}$$

selecting active mode, high capture rate, complex processing model, and continuous transmission. This policy tests whether energy constraints represent genuine system limitations or merely impose soft penalties. A well-designed energy harvesting system should experience catastrophic failures under this policy during periods of insufficient solar irradiance, validating that energy management is non-trivial.

Battery Threshold Policy

The battery threshold policy implements a simple reactive control strategy based on the current state of charge. The policy partitions the battery capacity into discrete regions and selects pre-defined action templates based on the observed $\mathrm{SoC}(t)$. Specifically, the policy applies the following decision rules:

Sleep Mode Selection. The sleep mode is determined by:

$$a_{\text{sleep}}(t) = \begin{cases} 2 & \text{if } \mathrm{SoC}(t) < 0.15 \\ 1 & \text{if } 0.15 \leq \mathrm{SoC}(t) < 0.30 \\ 0 & \text{if } \mathrm{SoC}(t) \geq 0.30 \end{cases} \tag{5.9}$$

forcing deep sleep during critical battery levels, idle mode during moderate scarcity, and active operation when energy is sufficient.

Capture Mode Selection. Image capture intensity adapts to battery availability:

$$a_{\text{cap}}(t) = \begin{cases} 0 & \text{if SoC}(t) < 0.20 \\ 1 & \text{if } 0.20 \leq \text{SoC}(t) \leq 0.60 \\ 2 & \text{if SoC}(t) > 0.60 \end{cases} \tag{5.10}$$

suspending capture during low energy, operating conservatively during moderate states, and exploiting abundant energy opportunistically.

Processing Mode Selection. Processing model selection follows similar threshold-based logic:

$$a_{\text{proc}}(t) = \begin{cases} 0 & \text{if SoC}(t) < 0.25 \\ 1 & \text{if } 0.25 \leq \text{SoC}(t) \leq 0.65 \\ 2 & \text{if SoC}(t) > 0.65 \end{cases} \tag{5.11}$$

prioritizing simple models during energy constraints and enabling complex inference when energy permits.

Transmission Control. Transmission is enabled conservatively:

$$a_{\text{tx}}(t) = \begin{cases} 0 & \text{if SoC}(t) < 0.20 \\ 1 & \text{if SoC}(t) \geq 0.20 \end{cases} \tag{5.12}$$

recognizing the relatively high energy cost of wireless communication.

This policy provides a reasonable baseline demonstrating that simple battery-aware control yields non-trivial performance, while remaining agnostic to workload dynamics and future energy availability.

Smart Heuristic Policy

The smart heuristic represents a sophisticated hand-crafted policy incorporating battery state, buffer occupancy, and solar forecast information. This policy embodies substantial domain expertise and serves as a strong baseline against which to evaluate the necessity of learned control.

State-Aware Threshold Adaptation. Unlike the fixed threshold policy, the smart heuristic dynamically adjusts decision boundaries based on predicted solar availability. Let $\hat{E}_{\text{forecast}}(t)$ denote the estimated harvested energy over the next 12 time steps,

computed from the solar irradiance forecast. The policy defines adaptive thresholds:

$$\text{SoC}_{\text{low}}(t) = \begin{cases} 0.25 & \text{if } \hat{E}_{\text{forecast}}(t) > 500 \text{ W/m}^2 \\ 0.40 & \text{otherwise} \end{cases} \tag{5.13}$$

$$\text{SoC}_{\text{high}}(t) = \begin{cases} 0.55 & \text{if } \hat{E}_{\text{forecast}}(t) > 500 \text{ W/m}^2 \\ 0.70 & \text{otherwise} \end{cases} \tag{5.14}$$

adopting more conservative thresholds when solar conditions are unfavorable, effectively implementing predictive energy management.

Buffer-Aware Capture Control. The policy modulates capture activity to prevent buffer overflow while maintaining responsiveness. The capture decision incorporates both battery state and raw buffer occupancy $Q_{\text{raw}}(t)$:

$$a_{\text{cap}}(t) = \begin{cases} 0 & \text{if SoC}(t) < \text{SoC}_{\text{low}}(t) \text{ or } \frac{Q_{\text{raw}}(t)}{Q_{\text{raw}}^{\max}} > 0.85 \\ 1 & \text{if } \frac{Q_{\text{raw}}(t)}{Q_{\text{raw}}^{\max}} > 0.70 \\ 2 & \text{if SoC}(t) > \text{SoC}_{\text{high}}(t) \text{ and } \hat{E}_{\text{forecast}}(t) > 400 \\ 1 & \text{otherwise} \end{cases} \tag{5.15}$$

This logic prevents buffer saturation during processing bottlenecks while opportunistically increasing capture rates during favorable conditions.

Backlog-Clearing Processing. Processing decisions prioritize clearing accumulated backlogs in the raw image buffer:

$$a_{\text{proc}}(t) = \begin{cases} 0 & \text{if SoC}(t) < \text{SoC}_{\text{low}}(t) \text{ or } \frac{Q_{\text{raw}}(t)}{Q_{\text{raw}}^{\max}} < 0.10 \\ 2 & \text{if } \frac{Q_{\text{raw}}(t)}{Q_{\text{raw}}^{\max}} > 0.60 \text{ and SoC}(t) > 0.50 \\ 1 & \text{if } \frac{Q_{\text{raw}}(t)}{Q_{\text{raw}}^{\max}} > 0.60 \\ 2 & \text{if SoC}(t) > \text{SoC}_{\text{high}}(t) \\ 1 & \text{otherwise} \end{cases} \tag{5.16}$$

selecting aggressive processing when backlogs accumulate and energy permits, while conserving resources when buffers are empty or energy is scarce.

Buffer-Prioritized Transmission. Transmission activation balances energy conservation with buffer management:

$$a_{\text{tx}}(t) = \begin{cases} 0 & \text{if } \text{SoC}(t) < 0.20 \\ 1 & \text{if } \frac{Q_{\text{proc}}(t)}{Q_{\text{proc}}^{\max}} > 0.40 \\ 1 & \text{if } \frac{Q_{\text{proc}}(t)}{Q_{\text{proc}}^{\max}} > 0.15 \text{ and } \text{SoC}(t) > 0.35 \\ 0 & \text{otherwise} \end{cases} \tag{5.17}$$

activating transmission preemptively as processed buffers fill, preventing data loss from overflow.

The smart heuristic policy demonstrates that carefully engineered rules incorporating multiple state variables and predictive information can achieve competitive performance. However, the manual threshold tuning process is labor-intensive and may not generalize across deployment scenarios with different energy availability patterns or workload characteristics.

5.7.2 Model Predictive Control Baselines

Model predictive control represents a principled optimization-based approach to sequential decision making under constraints. Unlike heuristics, MPC explicitly optimizes a cost function over a finite planning horizon, offering theoretical optimality guarantees when the system model is accurate.

MPC Problem Formulation

At each time step t, the MPC controller solves a finite-horizon optimal control problem:

$$\begin{aligned} J(\mathbf{s}, \mathbf{a}) = &- w_{\text{delivery}} \cdot r_{\text{delivery}}(\mathbf{s}, \mathbf{a}) + w_{\text{energy}} \cdot \max\{0, 0.3 - \text{SoC}\} \\ &+ w_{\text{safety}} \cdot \mathbb{I}_{B<B_{\text{threshold}}} + w_{\text{smoothness}} \cdot \|\mathbf{a} - \mathbf{a}_{\text{prev}}\|_1 \end{aligned} \tag{5.18}$$

where h denotes the planning horizon, $J(\cdot)$ represents the stage cost, $f(\cdot)$ is the system dynamics model, and $\mathbf{w}(\tau)$ captures stochastic disturbances. The optimization yields an action sequence $\{\mathbf{a}^*(\tau)\}_{\tau=t}^{t+h-1}$, of which only the first action $\mathbf{a}^*(t)$ is executed, following the receding horizon principle.

Cost Function Design

The MPC stage cost penalizes energy consumption and battery depletion while rewarding event delivery:

$$\begin{aligned} J(\mathbf{s}, \mathbf{a}) = &- w_{\text{delivery}} \cdot r_{\text{delivery}}(\mathbf{s}, \mathbf{a}) + w_{\text{energy}} \cdot \max\{0, 0.3 - \text{SoC}\} \\ &+ w_{\text{safety}} \cdot \mathbb{I}_{B<B_{\text{threshold}}} + w_{\text{smoothness}} \cdot \|\mathbf{a} - \mathbf{a}_{\text{prev}}\|_1 \end{aligned} \tag{5.19}$$

where $w_{\text{delivery}} = 120$, $w_{\text{energy}} = 15$, $w_{\text{safety}} = 1500$, and $w_{\text{smoothness}} = 0.5$ are manually tuned weights. The delivery reward $r_{\text{delivery}}(\mathbf{s}, \mathbf{a})$ equals the number of images transmitted when an important event is active. The energy term increases costs when SoC falls below 30%, while the safety term heavily penalizes predicted battery threshold violations. The smoothness term discourages rapid action changes that may induce unnecessary subsystem transitions.

System Dynamics Model

The MPC controller employs a simplified deterministic model of system evolution. The battery dynamics follow:

$$B(\tau + 1) = \min\{B_{\max}, \max\{0, B(\tau)(1 - \lambda_{\text{leak}}) + \hat{E}_{\text{harv}}(\tau) - \hat{E}_{\text{total}}(\tau, \mathbf{a}(\tau))\}\} \tag{5.20}$$

where $\hat{E}_{\text{harv}}(\tau)$ represents the predicted harvested energy and $\hat{E}_{\text{total}}(\tau, \mathbf{a}(\tau))$ denotes the estimated energy consumption for action $\mathbf{a}(\tau)$, computed using measured subsystem power profiles.

Buffer evolution is modeled deterministically based on capture, processing, and transmission rates:

$$Q_{\text{raw}}(\tau + 1) = \min\{Q_{\text{raw}}^{\max}, Q_{\text{raw}}(\tau) + n_{\text{captured}}(\mathbf{a}(\tau)) - n_{\text{processed}}(\mathbf{a}(\tau))\} \tag{5.21}$$

$$Q_{\text{proc}}(\tau + 1) = \min\{Q_{\text{proc}}^{\max}, Q_{\text{proc}}(\tau) + n_{\text{processed}}(\mathbf{a}(\tau)) - n_{\text{transmitted}}(\mathbf{a}(\tau))\} \tag{5.22}$$

where n_{captured}, $n_{\text{processed}}$, and $n_{\text{transmitted}}$ depend on the selected action components and current buffer states according to the system model presented in Chapter 3.

Solar Forecast Generation

Since accurate long-horizon solar forecasting is challenging, the MPC baseline employs a simple persistence forecast:

$$\hat{E}_{\text{harv}}(\tau) = E_{\text{harv}}(t) \quad \forall \tau \in [t, t + h) \tag{5.23}$$

assuming that the current irradiance level persists over the planning horizon. While simplistic, this approach reflects the limited predictive information available in practical deployments without sophisticated meteorological forecasting systems.

Optimization Solver

Due to the combinatorial nature of the discrete action space and the computational constraints of embedded deployment, we employ sampling-based optimization rather than exact methods. At each time step, the controller:

1. Generates $N_{\text{samples}} = 120$ candidate action sequences by randomly sampling actions at each step within the horizon
2. Evaluates the cost of each sequence using the system dynamics model
3. Selects the sequence with minimal cost
4. Executes the first action of the optimal sequence

To improve initial guesses, we implement a warm-start strategy that initializes one candidate sequence based on current battery state: aggressive actions when SoC exceeds 60%, moderate actions when SoC is between 30% and 60%, and conservative actions when SoC falls below 30%.

MPC Variants

We evaluate four MPC variants with different planning horizons to analyze the trade-off between solution quality and computational cost:

MPC-Greedy ($h = 1$). This variant performs single-step optimization, reducing to a myopic greedy policy. While computationally efficient, it cannot anticipate future energy availability or workload demands.

MPC-Short ($h = 10$). Planning over 10 time steps (10 minutes with $\Delta t = 1$ minute) provides moderate lookahead while maintaining reasonable computational requirements. This horizon is sufficient to capture short-term variations in solar irradiance.

MPC-Long ($h = 50$). Extended planning over 50 steps enables longer-term strategic decisions, potentially exploiting knowledge of approaching high-solar periods. However, the accumulation of model error over extended horizons may degrade solution quality.

MPC-Adaptive. This variant dynamically adjusts the horizon based on battery state:

$$h(t) = \begin{cases} 5 & \text{if SoC}(t) < 0.25 \\ 25 & \text{if } 0.25 \leq \text{SoC}(t) \leq 0.60 \\ 50 & \text{if SoC}(t) > 0.60 \end{cases} \tag{5.24}$$

employing shorter horizons during energy scarcity to focus on immediate survival, and longer horizons during energy abundance to optimize longer-term performance.

The diversity of MPC configurations allows investigation of whether planning horizon length significantly affects performance, and whether the fundamental MPC approach is competitive with model-free reinforcement learning for this application.

5.7.3 Alternative Reinforcement Learning Baseline: Deep Q-Network

To assess whether the superior performance of PPO stems from the specific algorithm choice or from the general applicability of reinforcement learning, we implemented a Deep Q-Network (DQN)[37] baseline representing an alternative RL paradigm.

DQN Architecture

The DQN agent learns a state-action value function $Q(s, a; \theta)$ approximated by a feed-forward neural network. The network architecture consists of:

- Input layer: State vector of dimension d_s (matching the observation space)
- Hidden layer 1: 256 units with ReLU activation
- Hidden layer 2: 256 units with ReLU activation
- Output layer: $|\mathcal{A}|$ units (one per discrete action), linear activation

This architecture contains 150,199 parameters, similar in scale to the PPO policy network. The network outputs Q-values for all possible actions given the current state, from which the agent selects actions via ϵ-greedy exploration:

$$a(t) = \begin{cases} \arg\max_{a' \in \mathcal{A}} Q(s(t), a'; \theta) & \text{with probability } 1 - \epsilon(t) \\ \text{uniform random from } \mathcal{A} & \text{with probability } \epsilon(t) \end{cases} \tag{5.25}$$

where $\epsilon(t)$ decays linearly from 1.0 to 0.01 over the first 50% of training episodes.

Training Procedure

The DQN agent is trained using experience replay and target networks following the standard DQN algorithm. Key hyperparameters include:

- Replay buffer capacity: 100,000 transitions
- Minibatch size: 64
- Learning rate: $\alpha = 10^{-4}$ with Adam optimizer
- Discount factor: $\gamma = 0.99$
- Target network update frequency: Every 1,000 steps
- Training episodes: 5,000

At each training step, the agent samples a minibatch from the replay buffer and updates the Q-network parameters by minimizing the temporal difference error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right] \tag{5.26}$$

where θ^- denotes the target network parameters, updated periodically from the online network.

Action Space Representation

A critical design choice for DQN concerns action space representation. The composite action space $\mathcal{A} = \mathcal{A}_{\text{sleep}} \times \mathcal{A}_{\text{cap}} \times \mathcal{A}_{\text{proc}} \times \mathcal{A}_{\text{tx}}$ contains $|\mathcal{A}| = 3 \times 3 \times 3 \times 2 = 54$ discrete actions for Jetson and $3 \times 3 \times 2 \times 2 = 36$ for Raspberry Pi (due to limited processing options). We represent each composite action as a single discrete choice, requiring the Q-network to output 54 (or 36) Q-values corresponding to all valid action combinations.

This flat representation is straightforward to implement but may be sample-inefficient compared to factored representations that exploit action structure. However, for fair comparison with PPO, we maintain consistent state and action representations across both algorithms.

DQN Evaluation Protocol

The DQN agent is trained exclusively on the Raspberry Pi platform, as preliminary experiments indicated training instability on Jetson due to the larger action space and higher variance in episode returns. The trained DQN policy is evaluated using the same protocol as other baselines: 100 episodes with fixed random seeds, measuring mean reward, delivery rate, completion rate, and computational metrics.

5.7.4 Safety Wrapper Mechanism

All baseline policies, including the RL agents, are evaluated both with and without a safety wrapper that enforces energy feasibility constraints at runtime. This wrapper serves two purposes: (1) preventing catastrophic system failures during evaluation, and (2) assessing the robustness of learned policies to deployment constraints.

Wrapper Implementation

The safety wrapper operates as a post-processing layer that intercepts the action selected by the base policy and potentially overrides it with a safer alternative. At each time step, before executing action $\mathbf{a}(t)$ proposed by the policy, the wrapper performs the following checks:

Energy Feasibility Check. The wrapper estimates the energy cost of the proposed action:

$$\hat{E}_{\text{cost}}(\mathbf{a}(t)) = \hat{E}_{\text{cap}}(a_{\text{cap}}(t)) + \hat{E}_{\text{proc}}(a_{\text{proc}}(t)) + \hat{E}_{\text{tx}}(a_{\text{tx}}(t)) + \hat{E}_{\text{standby}}(a_{\text{sleep}}(t)) \quad (5.27)$$

using pre-measured energy consumption profiles for each subsystem mode.

Safety Override Condition. If executing the proposed action would deplete the battery below the critical threshold:

$$B(t) - \hat{E}_{\text{cost}}(\mathbf{a}(t)) < B_{\min} \quad (5.28)$$

the wrapper overrides the action with a conservative fallback. The override logic follows:

1. If $\text{SoC}(t) < 0.25$: Force deep sleep mode $\mathbf{a}(t) \leftarrow (2, 0, 0, 0)$
2. Else if $\text{SoC}(t) < 0.35$: Be active with minimal activity capture in low rate, process with the simple model and send $\mathbf{a}(t) \leftarrow (0, 1, 1, 1)$
3. Otherwise: Allow proposed action

Gradual Degradation. Rather than abruptly halting all operations, the wrapper implements a gradual degradation strategy that progressively restricts expensive operations as energy becomes scarce. This approach maintains partial functionality even during energy crises, maximizing the probability of handling critical events while preventing complete system failure.

Wrapper Statistics

During evaluation, the wrapper tracks intervention statistics including:

- Number of action overrides per episode
- Distribution of override severity (minor modifications vs. complete shutdowns)
- Energy saved through interventions
- Events missed due to forced conservation

These statistics provide insight into how frequently policies propose unsafe actions and the performance cost of enforcing safety constraints.

5.7.5 Baseline Summary and Expected Performance Ordering

The diverse set of baselines enables multi-faceted evaluation of the proposed PPO approach:

Sanity Checks. Random and Always Sleep policies establish lower bounds and verify environment correctness.

Energy Management Necessity. Maximum Throughput tests whether energy constraints are binding.

Simple Reactivity. Battery Threshold demonstrates that basic battery-aware control is non-trivial.

Domain Expertise. Smart Heuristic represents carefully engineered domain knowledge.

Model-Based Optimization. MPC variants[17] evaluate whether accurate system models enable superior control.

Alternative RL. DQN assesses algorithm-specific vs. paradigm-level advantages of RL.

Based on the design characteristics, we hypothesize the following performance ordering:

$$\begin{aligned} J(\mathbf{s}, \mathbf{a}) = &- w_{\text{delivery}} \cdot r_{\text{delivery}}(\mathbf{s}, \mathbf{a}) + w_{\text{energy}} \cdot \max\{0, 0.3 - \text{SoC}\} \\ &+ w_{\text{safety}} \cdot \mathbb{I}_{B<B_{\text{threshold}}} + w_{\text{smoothness}} \cdot \|\mathbf{a} - \mathbf{a}_{\text{prev}}\|_1 \end{aligned} \tag{5.29}$$

where the last three policies are expected to fail frequently in unwrapped configurations. The subsequent experimental evaluation validates these hypotheses and quantifies performance differences rigorously.

Chapter 6

Measurements and Hardware

6.1 System Architecture: Asynchronous Edge-to-Cloud Pipeline

6.1.1 Overview

In this project, we implemented a decoupled, asynchronous processing pipeline on the NVIDIA Jetson Nano. The system addresses the resource constraints of Edge IoT devices by employing a **Hybrid Storage Strategy**. To mitigate I/O latency bottlenecks, high-bandwidth data (images) operates within a volatile RAM filesystem, while critical state data remains on persistent physical storage.

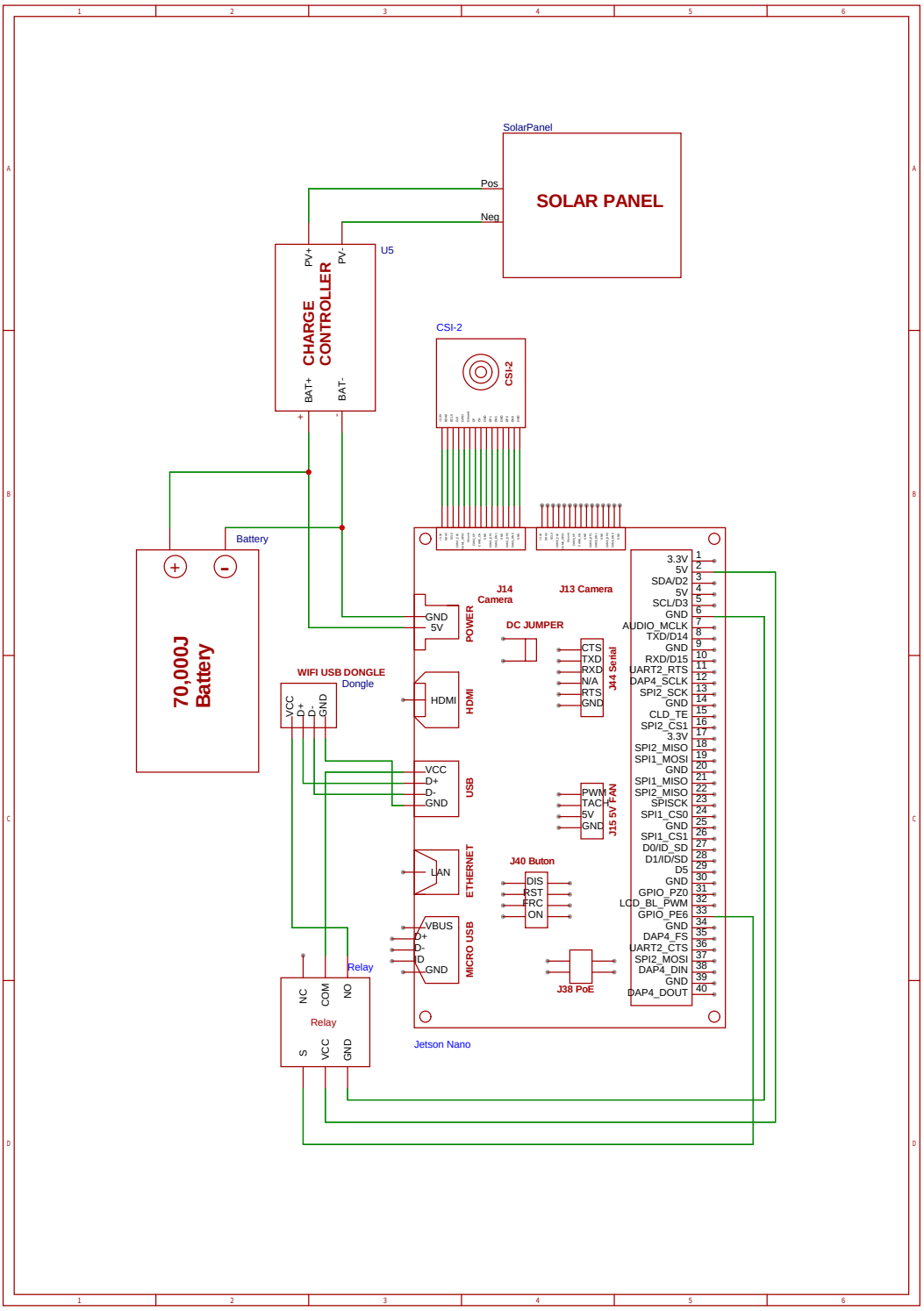

Figure 6.1: System Hardware Schematic.

6.1.2 Hybrid Data Persistence Strategy

The system divides data into two categories based on volatility and bandwidth requirements.

Volatile Storage: The Role of `/dev/shm`

To achieve high-frequency capture rates without SD card wear or write-latency, image artifacts are stored in `/dev/shm`.

- **Technical Definition:** `/dev/shm` is a temporary file storage filesystem implementation of the POSIX Shared Memory standard. Unlike standard disk partitions, files written here reside entirely in the device's Random Access Memory (RAM).
- **Performance Benefit:** This eliminates the mechanical or flash-controller latency associated with writing to the SD card, allowing for instant serialization of raw image frames.
- **Volatility Constraints:** As RAM is volatile, all data in this directory is lost upon system reboot. Therefore, it is used exclusively for the *Circular Image Buffer*, which is designed to be transient.

Persistent Storage: Logs and State

To ensure the system can recover its state after a power cycle, low-bandwidth metadata is written to the physical disk (SD Card).

- **State Directory (`./state/`):** Contains the JSON pointer files. This ensures that if the device reboots, the *Capture*, *Inference*, and `Upload` services recall exactly where they left off (e.g., $I_{cap} = 5043$).
- **Logs Directory (`./logs/`):** Stores the daily CSV historical records. Since these files are small (text-only), the I/O penalty is negligible, but their persistence is critical for data integrity.

6.1.3 Inter-Process Communication (IPC)

Services synchronize via lightweight JSON state files located in the persistent `./state/` directory. To prevent corruption during sudden power loss, these files are updated using an **Atomic Write-Replace** strategy.

- `capture_pointer.json`: Stores I_{cap} (Total images captured).
- `inference_pointer.json`: Stores I_{inf} (Total images processed).
- `upload_pointer.json`: Stores I_{up} (Total images uploaded).

6.1.4 Operational Workflow: The Cascading Dependency

The system operates as a Cascading State Machine governed by the inequality:

$$I_{up} \leq I_{inf} \leq I_{cap} \tag{6.1}$$

Stage 1: Acquisition (Capture Service)

Role: The Producer (Leader).

1. **Date Resolution:** Checks system date. If a new day is detected, it atomically initializes a new CSV file in `./logs/`.
2. **Capacity Management:** Before writing to RAM, the service invokes the *Buffer Manager*. If the current buffer size has reached its **800 MB allocation**, the oldest image is evicted to make room for the incoming frame.
3. **Capture:** Serializes the raw image frame directly to the volatile RAM buffer (`/dev/shm`).
4. **State Update:** Appends metadata to the CSV on disk and atomically updates `capture_pointer.json`.

Stage 2: Processing (Inference Service)

Trigger: Active when $I_{inf} < I_{cap}$ (Reads pointers from disk).

- **Logic:** Reads metadata at index I_{inf} and looks for the corresponding image in `/dev/shm`.
- **Hit:** Executes inference model, logs result to the CSV on disk.
- **Miss (Evicted):** Logs `Inference_Result = "SKIPPED_MISSING"` if the image was removed by the Janitor before processing.

Stage 3: Offloading (Upload Service)

Role: The Finalizer.
Trigger: Active when $I_{up} < I_{inf}$.

- **Adaptive Payload Strategy:**
  - **Case A (Result "SKIPPED"):** No transmission; Index incremented.
  - **Case B (Valid Result + Image Present):** Uploads full payload (Image from RAM + Metadata from Disk).
  - **Case C (Valid Result + Image Missing):** Uploads Metadata Only (Classification Label).

6.1.5 Stability & Resource Management

Fixed-Capacity Resource Management (The "Buffer Manager")

To ensure the system remains within a predictable memory footprint, `/dev/shm` is managed as a **Fixed-Size Circular Buffer** capped at **800 MB**.

- **Trigger Condition:** The manager is invoked every time a new capture is requested.
- **Logic (FIFO Eviction):** The system monitors the total size of image artifacts in the volatile directory. Once the 800 MB limit is reached, the system performs a **one-in, one-out** replacement: the oldest image file is deleted immediately before the new image is written.
- **Impact:** This ensures the capture service never exceeds its allocated RAM slice, protecting the OS from memory exhaustion while maintaining the most recent N frames for downstream processing.

Transactional Integrity

- **Atomic State Updates:** JSON state files are never written directly. The system writes to a temporary file (`state.tmp`) and performs an atomic rename (`os.replace`). This ensures the index is never corrupted, even if power is cut mid-write.

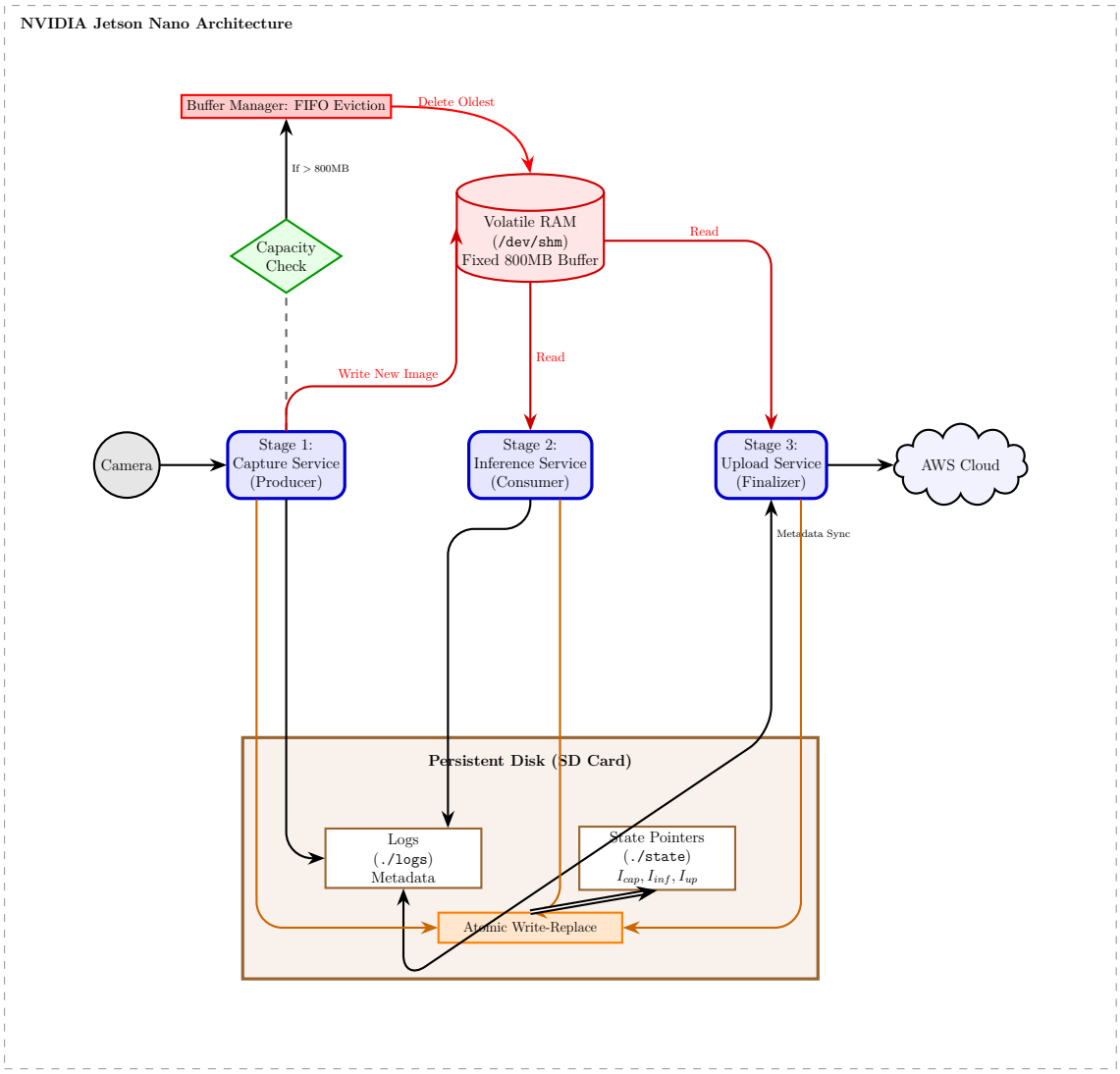

Figure 6.2: System Architecture: High-Consistency Asynchronous Pipeline.

6.2 Energy Characterization & Resource Profiling

To enable effective Reinforcement Learning, we first characterize the physical behavior of the camera hardware (Section 2.1), followed by a detailed profiling of the specific software services, beginning with the Capture Service (Section 2.2).

6.2.1 Statistical Methodology: Defining "Energy Risk"

To characterize the reliability of the system for Reinforcement Learning, we treat energy consumption not as a single scalar value, but as a probabilistic distribution. For every service (Capture, Inference, Upload), we calculate the **Upper Confidence Bound (UCB)** using the **3-Sigma Rule**:

$$E_{risk} = \mu + 3\sigma \tag{6.2}$$

Where μ is the mean energy cost and σ is the standard deviation.

- **Interpretation:** We are 99.7% confident that any given operation will cost less than E_{risk}.
- **RL Application:** The agent uses μ for expected value calculations, but uses σ (specifically the Coefficient of Variation) to detect "Risky States" where thermal instability might cause battery drain anomalies.

6.2.2 Pipeline Stage 1: Capture Service (Image Acquisition)

The Capture Service constitutes the sensory front-end of the pipeline. To construct a high-fidelity energy model, we instrumented the Jetson Nano's `INA3221` power monitor rail (`in_power0_input`) at a sampling frequency of $f_s = 100$ Hz.

A critical distinction in this model is between the one-time **Software Initialization** (E_{init}) and the recurring operational costs of the camera hardware.

Experimental Profiling & Definitions

We utilized a "Hardware-in-the-Loop" profiling protocol to isolate the following energy components:

- **Software Initialization (E_{init}):** The "Cold Start" penalty incurred when loading the Python interpreter and GStreamer libraries. This cost is incurred only once when the service transitions from an inactive state to an active state.
- **Hardware Startup ($E^H_{\mathbf{start}}$):** The energy required to wake the CMOS sensor and stabilize the Image Signal Processor (ISP).
- **Steady-State Operation:** The recurring costs associated with active image capture and inter-frame standby power.

Mode-Specific Architecture & Energy Models

The system employs two distinct architectural strategies—*Process-per-Image* versus *Persistent Streaming*—governed by the inter-arrival time of capture events.

Design Rationale: Latency vs. Energy Trade-off The choice of strategy is dictated by the comparison between the cost of maintaining the camera in a "Standby" state (P_{stby}) and the cost of repeatedly restarting the system ($E_{init} + E^H_{\text{start}}$).

- **In Low Rate scenarios ($N = 6$):** The inter-capture delay is significant ($t_{\text{delay}} \approx$ 8.5s). Keeping the camera active would incur a massive standby penalty ($P_{\text{stby}} \times$

$8.5s \approx 7$ Joules per interval). It is therefore energetically optimal to completely shut down the service between frames, accepting the restart cost (≈ 1.1 Joules) to save energy.

- **In High Rate scenarios ($N = 120$):** The delay is minimal (0.42s). Shutting down would not only be energetically inefficient (due to the repeated high-cost initialization) but also operationally infeasible, as the hardware wake-up latency would exceed the available time window. Thus, "Persistent Streaming" is required.

Low Rate Mode: Process-per-Image In this mode, the system captures a sparse sequence of images. To minimize idle power during the long delays, the system spawns a fresh process for every frame. Consequently, the active processing cost is incurred repeatedly.

Energy Model: The total energy is the sum of the active processing energy per image and the fixed logging overhead for the batch:

$$E_{low} = (N \cdot e^L_{\text{active}}) + E^L_{\text{log}} \tag{6.3}$$

Where e^L_{active} aggregates the initialization, capture, and termination costs per frame.

High Rate Mode: Persistent Streaming In this mode, the system maximizes throughput by maintaining an open camera stream to capture $N = 120$ images within a fixed 60-second epoch. The hardware startup cost is incurred only once, followed by a sequence of captures separated by short standby intervals.

The inter-capture delay ($t_{\text{delay}} \approx 0.42$s) was specifically calibrated to generate a total active capture time of approximately 50.4 seconds (120×0.42). This design leaves a safety margin of ≈ 9.6 seconds within the epoch. This buffer is critical for system stability: if the Jetson Nano experiences heavy computational load, the capture process can expand into this margin without overrunning the epoch or delaying subsequent tasks.

Energy Model: The total energy consists of the hardware startup, the cumulative capture energy, the standby energy dissipated during intervals, and the logging overhead:

$$E_{high} = E^H_{\text{start}} + N \cdot e^H_{\text{cap}} + (N-1)e^H_{\text{stby}} + E^H_{\text{log}} \tag{6.4}$$

Where:

- E^H_{start}: Initial hardware wake-up energy.
- e^H_{cap}: Energy to capture a single frame in streaming mode.

- e^H_{stby}: Energy consumed during the inter-frame delay ($P_{\text{stby}} \times t_{\text{delay}}$).

Summary of Empirical Parameters

Table 6.1 presents the measured parameters derived from the profiling rounds, mapped to the model symbols.

Table 6.1: Capture Service Energy Parameters

Parameter	Symbol	Mean	Std Dev	CV (%)
Global Constants				
Software Initialization (mJ)	E_{init}	857.25	370.01	42.1
Hardware Startup (mJ)	E^H_{start}	200.00	–	–
Epoch Duration (s)	T_{epoch}	60.00	–	–
Low Rate Mode ($N = 6$)				
Active Energy / Image (mJ)	e^L_{active}	1172.15	64.89	5.5
Logging Overhead (mJ)	E^L_{log}	11.93	11.20	93.9
High Rate Mode ($N = 120$)				
Standby Power (mW)	P_{stby}	830.48	61.26	7.4
Capture Energy / Image (mJ)	e^H_{cap}	10.49	25.12	239.5
Logging Overhead (mJ)	E^H_{log}	419.37	57.11	13.6
Inter-Capture Delay (s)	t_{delay}	0.42	–	–

6.2.3 Pipeline Stage 2: Inference Service

The Inference Service represents the computational core of the pipeline, responsible for object detection and classification. To characterize its energy footprint, we analyzed distinct model architectures (e.g., "Complex" vs. "Simple") to understand the cost of precision.

To ensure statistical robustness, **Core Processing** and **Initialization** costs were aggregated across the entire experimental dataset. Conversely, batch-dependent overheads (such as **Logging**) are reported for saturation batch sizes to illustrate behavior under steady-state load.

Component-Level Energy Analysis

The energy consumption of the inference stage is composed of three distinct phases: Service Initialization, Core Processing (Resize + Inference), and Data Logging.

1. Service Initialization (E_{init}) Before processing begins, the service must load the interpreter. This "Cold Start" penalty is consistent across all trials ($\approx 23 - 25$ mJ). This cost is incurred only when the service starts from an inactive state.

2. Core Processing ($e_{\mathbf{proc}}$) This phase scales linearly with the number of images processed. Table 6.2 presents the aggregated statistics:

- **Preprocessing (Resize):** The cost to resize raw input images (≈ 16 mJ/image).
- **Neural Inference:** The dominant energy sink. The Complex model consumes ≈ 137.34 mJ per image, while the Simple model reduces this to ≈ 43.75 mJ.

3. Logging Overhead ($E_{\mathbf{log}}$) After processing a batch, the system serializes results. While it has high variance due to I/O buffering, its impact on the final budget is minor when amortized over large batches.

Table 6.2: Inference Core Energy Profile

Operation	**Simple Model**		**Complex Model**	
	Mean (mJ)	**CV (%)**	**Mean (mJ)**	**CV (%)**
Image Resizing	15.68	9.6	16.24	5.0
Neural Inference	43.75	3.8	137.34	2.7
Total Core ($e_{\mathbf{proc}}$)	**59.42**	**2.1**	**153.58**	**2.8**

Formal Energy Model

Based on these measurements, the total processing energy (E_t^{proc}) for a batch of images is defined as the sum of the per-image core cost and the fixed logging overhead:

$$E_t^{\text{proc}} = N_t^{\text{proc}} \cdot e_{\text{proc}} + E_{\log} \tag{6.5}$$

Where:

- N_t^{proc} is the number of images processed in the current batch.
- e_{proc} is the variable energy cost per image, dependent on the selected model architecture (e.g., 59.42 mJ or 153.58 mJ).
- $E_{\log}$ is the fixed batch logging overhead.

If the system was in an idle state prior to processing, an additional fixed cost (E_{init}) is added to the total energy consumed to account for the "Cold Start" penalty.

Efficiency Trade-off

The profiling data reveals that switching from the Complex to the Simple model reduces the per-image energy cost by approximately **61.3%**. The system objective is to dynamically select the model that balances this energy saving against the required classification accuracy.

6.2.4 Pipeline Stage 3: Transmission Service (Offloading)

The Transmission Service manages the upload of inference metadata to AWS DynamoDB and binary image data to AWS S3. Unlike the computational stages, the energy cost of wireless transmission is heavily influenced by network variability. To construct a deterministic energy model, we implemented strict hardware and software controls to isolate the cost of data transfer from environmental noise.

Experimental Control & Methodology

To ensure high-fidelity measurements, we introduced three layers of experimental control:

1. Hardware Power Gating (Relay Control) Standard "Sleep" modes for Wi-Fi dongles often exhibit significant leakage current. To eliminate this, we utilized a solid-state relay (controlled via GPIO Pin 33) to physically isolate the Wi-Fi dongle's power supply. The dongle is powered ($V_{dongle} = 5V$) only during active transmission windows, ensuring zero energy consumption during idle periods.

2. Network Traffic Shaping Wi-Fi power consumption fluctuates with available bandwidth. To standardize the transmission power state, we enforced a software bandwidth limit of **800 kbit/s** using the Linux Traffic Control (`tc`) utility.

- **Effective Throughput:** Empirical validation confirmed that this hard limit resulted in a stable effective throughput of ≈ 620 kbit/s.
- **Energy Implication:** By capping the upload speed below the network's theoretical maximum, we forced the radio into a linear transmission state where energy cost is directly proportional to payload size ($E \propto$ size).

3. Batch Sizing Strategy We analyzed two transmission modes with distinct batching requirements:

- **Result-Only (Metadata):** We selected $N = 2000$ as the optimal saturation point. This size maximizes throughput by amortizing the SSL handshake overhead while ensuring the transmission completes within the system's operational time slot.
- **Image + Result (Payload):** Due to the high latency and energy cost of uploading binary image data, we fixed the batch size at $N = 5$ to prevent timeout errors.

Phase 1: Service Initialization (E_{tx_init})

Every transmission cycle begins with a "Cold Start" sequence: activating the relay, negotiating the Wi-Fi handshake, and initializing the AWS SDK. As shown in Table 6.3, the dominant cost is **Polling Connection** ($\approx$ 2059 mJ), which represents the energy spent waiting for the Linux networking stack to acquire an IP address via DHCP.

Table 6.3: Transmission Initialization Energy Breakdown

Initialization Phase	Mean Energy (mJ)	Std Dev (mJ)	CV (%)
Script Startup	787.87	94.89	12.0
Polling Connection (DHCP/Assoc)	2059.22	391.77	19.0
Traffic Shaping (Applying Limit)	160.76	50.04	31.1
AWS SDK Initialization	769.24	183.56	23.9
Total Initialization (E_{tx_init})	**3777.09**	**434.30**	**11.5**

Phase 2: Operational Energy Costs

We isolated the unit costs for uploading a single metadata record versus a full image. The metadata cost was derived exclusively from the **Result-Only** logs, while the image

cost was derived from the **Image + Result** logs.

Table 6.4: Transmission Unit Costs (Per Item)

Operation	Mean Energy (mJ)	Std Dev (mJ)	CV (%)
DynamoDB Write	4.05	0.66	16.2
S3 Image Upload	248.79	62.66	25.2
Logging	0.05	0.02	40.4

Formal Energy Model

The total energy for the Transmission Service is modeled as the sum of the payload transmission cost and the baseline power consumption of the Wi-Fi dongle. We define two separate equations based on the transmission type:

$$E_{\text{meta}} = N \cdot e_{\text{meta}} + P_{\text{dongle}} \cdot \Delta t \tag{6.6}$$

$$E_{\text{image}} = N \cdot e_{\text{image}} + P_{\text{dongle}} \cdot \Delta t \tag{6.7}$$

Where:

- N is the number of items in the batch.
- e_{meta} is the unit cost for Result-Only upload (≈ 4.10 mJ).
- e_{image} is the unit cost for Image + Result upload (≈ 252.89 mJ).
- P_{dongle} is the constant background power of the Wi-Fi dongle.

Variable Duration (Δt): The dongle consumes background power throughout the entire active window. The duration depends on the system state:

- **Continuous Operation:** If the system was already active, $\Delta t = 60$s (the full epoch duration).
- **Cold Start:** If initialization occurred, the effective transmission time is reduced by the startup overhead, resulting in $\Delta t \approx 53.51$s.

Additional State Costs: Two fixed costs are applied conditionally in the system logic, outside the core transmission equations: 1. **Initialization (E_{tx_init}):** Added when starting from a cold state (≈ 3777 mJ). 2. **Relay Switch-Off (E_{off}):** A fixed cost of ≈ 543.81 mJ is incurred specifically when the system decides to close the relay and power down the dongle.

6.2.5 Standby Energy

The Jetson Nano's energy consumption during periods of inactivity is characterized by three distinct modes. While Active and Idle power were measured using the internal rail, the Deep Sleep power was measured using an external USB Tester to ensure accurate board-level power sensing during the suspend state.

$$E_{standby} = P_{standby} \times \Delta t \tag{6.8}$$

Table 6.5: Jetson Nano Standby Power States

Mode	Condition	Power (W)
Active	System performing tasks	$1.745815 + P_{active\_work}$
Idle	System on, no active tasks	1.745815
Deep Sleep	Suspend to RAM	0.2625

6.3 Cross-Platform Validation: Secondary System Architecture

To demonstrate the robustness of the Reinforcement Learning (RL) agent, we evaluate its performance against a secondary, highly constrained hardware architecture. This configuration decouples sensing from computation, utilizing a multi-hop pipeline typical of remote environmental monitoring. The data for this validation is synthesized from two primary research works.

6.3.1 Research Context and Methodology Briefs

- Reference A: "Low Power Environmental Image Sensors for Remote Photogrammetry" [18]. This research establishes a 4D monitoring solution (3D modeling + temporal monitoring) for the Allier River in France. The system is designed to study river dynamics and vegetation growth in areas without grid power. It utilizes ESP32-Cam nodes to capture images with a 66% overlap required for Structure from Motion (SFM) 3D reconstruction. Images are sent via Wi-Fi to a Raspberry Pi relay, which offloads the data to a remote server using long-range GSM (cellular) links.

- Reference B: "Benchmarking Deep Learning Models for Object Detection on Edge Computing Devices" [8]. This study provides a comprehensive benchmark of state-of-the-art deep learning algorithms on resource-constrained hardware. It measures inference time, energy consumption, and accuracy (mAP) across the

Raspberry Pi and NVIDIA Jetson families. We utilize their Raspberry Pi 3 data to characterize our relay's inference stage.

6.3.2 Secondary Pipeline: Capture Service (ESP32-Cam)

The acquisition stage is performed by the ESP32-Cam. To maximize battery life (targeted at 1.5 years), the sensor utilizes a Deep Sleep strategy between capture events.

Energy Model

The energy for capture (E_{cap}) includes the hardware boot penalty and the energy required to stabilize the sensor and serialize N images.

$$E_{cap} = E_{init} + N \cdot e_{shoot} \tag{6.9}$$

Empirical Parameters:

- E_{init}: Total energy for the boot sequence and camera configuration (≈ 809.7 mJ).
- e_{shoot}: Energy consumed per individual image capture (≈ 111.8 mJ).
- **Throughput Constraint:** A single photo requires ≈ 0.33s. Within the defined safety window, the hardware can support up to $N_{max} \approx 160$ images per epoch.

6.3.3 Secondary Pipeline: Inference Service (Raspberry Pi 3)

The Raspberry Pi 3 serves as the relay's computational engine. Following the benchmarking methodology in [8], we select the **EfficientDet Lite** family to represent our model-switching logic. EfficientDet Lite 0 serves as the **Simple Model**, while EfficientDet Lite 1 serves as the **Complex Model**.

Table 6.6: Inference Core Energy Profile (Raspberry Pi 3)

Tier	Model Architecture	Energy per Image (mJ)	mAP (Accuracy)
Simple	EfficientDet Lite 0	1476.0	26.0
Complex	EfficientDet Lite 1	2448.0	31.0

6.3.4 Secondary Pipeline: Transmission Service (GSM Cloud Bridge)

The transmission energy (E_{tx}) accounts for the energy-dense task of moving binary data over cellular networks. This cost is characterized by high latency (≈ 11.88s per 921 KB image) and significant fixed overheads.

Conditional Energy Model

The GSM module requires a daily access fee (E_{GSM_fee}). We model this as a conditional cost incurred only once per 24-hour cycle ($\mathbb{I}_{day} = 1$ for the first transmission of the day, 0 otherwise).

$$E_{tx} = E_{session} + (\mathbb{I}_{day} \cdot E_{GSM\_fee}) + N \cdot e_{unit\_tx} + (P_{overhead} \cdot \Delta t) \tag{6.10}$$

Empirical Parameters:

- $E_{session}$: Fixed energy for local Wi-Fi session negotiation (230 mJ).
- E_{GSM_fee}: Daily fixed energy cost for cellular network access (190,000 mJ).
- e_{unit_tx}: Aggregated energy to move one image through the entire pipeline: ESP32 local send, RPi local receive, and GSM cloud upload ($\approx 56,515$ mJ).
- $P_{overhead}$: Power overhead for the Raspberry Pi Wi-Fi module in active mode (220 mW).

6.3.5 Baseline Power and Maintenance Constants

The following constants represent the baseline and maintenance power for the secondary system. Halt power for the Raspberry Pi 3 is sourced from official hardware documentation [27].

Table 6.7: Baseline Power and Maintenance Constants

Component State	Power (mW)	Source
Raspberry Pi 3 Base Power (Idle)	3240.0	Benchmarking Study
Raspberry Pi 3 (Active, Wi-Fi Disabled)	1835.0	Environmental Study
Raspberry Pi 3 Halt Mode	500.0	Official RPi Documentation
ESP32-Cam (Deep Sleep)	16.5	Environmental Study

[1]Based on the Pi 3B official halt current of 0.10A at 5V ($P = IV$).

6.3.6 Discussion: System Generalization

This secondary architecture presents a distinct energy profile. In the Jetson Nano system, inference is the dominant energy consumer. In contrast, the ESP32/RPi system is dominated by transmission (56,515 mJ per image via GSM vs. 2,448 mJ for complex inference). By validating the RL agent on this second system, we prove that the agent can adapt its policy whether the "Energy Bottleneck" is computational (Inference) or environmental (Transmission).

Chapter 7

Results and Discussion

This chapter presents a comprehensive analysis of the curriculum learning training results for the energy harvesting camera control system. The PPO agent was trained over 4,700 episodes spanning six curriculum stages, achieving significant performance improvements and demonstrating the effectiveness of the proposed approach.

7.1 Overview of Training Results

The curriculum learning approach successfully trained a PPO agent to manage the energy harvesting camera system, achieving a final average event delivery rate of 74.6% (averaged over the last 100 episodes). This represents a substantial improvement over the initial performance of 28.2%, demonstrating a gain of +46.3 percentage points.

Table 7.1 summarizes the overall training trajectory.

Table 7.1: Overall training performance comparison

Metric	**Initial (Ep 0-100)**	**Final (Ep 4600-4700)**	**Improvement**	**Best Episode**
Episode Reward	−104.13	+487.48	+591.61 (+568%)	1034.66
Delivery Rate	28.2%	74.6%	+46.3 pp	96.3%
Energy Efficiency	5.37 events/J	4.54 events/J	−0.83	—

Figure 7.1 (referenced from the provided training plot) illustrates the learning progression across all six curriculum stages, showing episode rewards, delivery rates, energy efficiency, policy loss, value loss, and policy entropy over the entire training run.

7.2 Stage-by-Stage Analysis

This section provides detailed analysis of each curriculum stage, examining performance metrics, learning dynamics, and key observations.

7.2.1 Stage 1: Tutorial Ultra Easy (Episodes 0–200)

Configuration: The initial stage provided highly favorable conditions to facilitate basic learning:

- Initial SoC: 99%
- Event probability $p_{\text{event}} = 0.05$
- Mean images per event $\mu_{\text{images}} = 10$
- Event timeout $\tau_{\text{event}} = 500$ steps
- Miss penalty $r_{\text{missed}} = -1.0$
- Entropy coefficient $c_S = 0.30$

Performance Metrics:

$$\text{Mean Episode Reward} = -107.76 \pm 51.01$$
$$\text{Mean Delivery Rate} = 28.1\% \quad (\text{max: } 56.2\%)$$
$$\text{Energy Efficiency} = 5.37 \text{ events/J}$$

Analysis: This stage served as the initial exploration phase. Despite highly favorable conditions, the agent achieved only moderate performance:

- The negative mean reward indicates the agent had not yet learned effective event handling strategies
- The 28.1% delivery rate shows basic operational capability but poor prioritization
- High entropy ($c_S = 0.30$) encouraged broad exploration of the action space
- The agent learned fundamental system mechanics: energy consumption, buffer management, and basic processing

7.2.2 Stage 2: Tutorial (Episodes 200–700)

Configuration: Difficulty increased through higher event frequency and workload:

- Initial SoC: 95%
- Event probability $p_{\text{event}} = 0.10$ (2× increase)
- Mean images per event $\mu_{\text{images}} = 20$ (2× increase)
- Event timeout $\tau_{\text{event}} = 300$ steps
- Entropy coefficient $c_S = 0.20$

Performance Metrics:

$$\begin{aligned}\text{Mean Episode Reward} &= -137.53 \pm 88.94\\ \text{Initial Reward (first 50)} &= -161.90\\ \text{Final Reward (last 50)} &= -57.70\\ \text{Within-Stage Improvement} &= +104.20\ (+64.4\%)\\ \text{Mean Delivery Rate} &= 29.1\% \quad (\text{max: } 65.0\%)\end{aligned}$$

Analysis: This stage marked the **first major learning breakthrough**:

- Substantial within-stage improvement (+64.4%) demonstrates effective learning
- Maximum delivery rate of 65.0% shows the agent discovered viable strategies
- High variance (±88.94) reflects active exploration and policy refinement
- Delivery rate improved from 26.8% to 34.5% within the stage
- Reduced entropy ($c_S = 0.20$) allowed increased exploitation of discovered strategies

The agent began learning to distinguish important from regular events and developed basic deadline-aware processing strategies.

7.2.3 Stage 3: Easy (Episodes 700–1500)

Configuration: Further increases in workload complexity:

- Initial SoC: 90%
- Event probability $p_{\text{event}} = 0.15$
- Mean images per event $\mu_{\text{images}} = 30$ (3× initial)
- Event timeout $\tau_{\text{event}} = 300$ steps
- Entropy coefficient $c_S = 0.18$

Performance Metrics:

$$\begin{aligned}\text{Mean Episode Reward} &= -27.96 \pm 158.92\\ \text{Mean Delivery Rate} &= 41.4\% \quad (\text{max: } 88.2\%)\\ \text{Energy Efficiency} &= 5.10 \text{ events/J}\end{aligned}$$

Analysis: This stage represents a **transition and consolidation period**:

- Highest variance across all stages (±158.92) indicates significant adaptation challenges
- Mean delivery rate improved to 41.4% despite within-stage performance fluctuation
- Peak performance of 88.2% demonstrates capability under favorable conditions
- The tripling of images per event significantly increased buffer management complexity
- Slight energy efficiency decrease (5.10 from 5.38) reflects increased system stress

The temporary within-stage performance dip suggests the difficulty increase was substantial, but the agent ultimately adapted through continued learning.

7.2.4 Stage 4: Medium (Episodes 1500–2500)

Configuration: Significant increases across multiple parameters:

- Initial SoC: 80%
- Event probability $p_{\text{event}} = 0.20$
- Mean images per event $\mu_{\text{images}} = 50$ (5× initial)
- Event timeout $\tau_{\text{event}} = 250$ steps
- Miss penalty $r_{\text{missed}} = -3.0$
- Entropy coefficient $c_S = 0.15$

Performance Metrics:

$$\begin{aligned} \text{Mean Episode Reward} &= +150.87 \pm 260.47 \\ \text{Initial Reward (first 50)} &= -29.06 \\ \text{Final Reward (last 50)} &= +103.36 \\ \text{Within-Stage Improvement} &= +132.42\ (+455.7\%) \\ \text{Mean Delivery Rate} &= 55.5\% \quad (\text{max: } 90.1\%) \end{aligned}$$

Analysis: This stage marks the **second major learning breakthrough**:

- Dramatic 455.7% within-stage improvement is the largest relative gain
- Mean reward transitioned into positive territory (+150.87)
- Delivery rate improved from 42.3% to 50.1% within the stage
- Successfully adapted to reduced battery capacity (80% SoC) and tighter deadlines
- Reduced entropy ($c_S = 0.15$) enabled focused exploitation of refined strategies

The substantial performance gains demonstrate the value of curriculum learning—direct training at this difficulty level would likely fail to converge.

7.2.5 Stage 5: Hard (Episodes 2500–3500)

Configuration: Approaching realistic deployment conditions:

- Initial SoC: 60%
- Event probability $p_{\text{event}} = 0.25$
- Mean images per event $\mu_{\text{images}} = 75$
- Event timeout $\tau_{\text{event}} = 200$ steps
- Miss penalty $r_{\text{missed}} = -3.5$
- Entropy coefficient $c_S = 0.12$

Performance Metrics:

$$\begin{aligned} \text{Mean Episode Reward} &= +323.82 \pm 304.27 \\ \text{Within-Stage Improvement} &= +83.51 \ (+26.9\%) \\ \text{Mean Delivery Rate} &= 65.9\% \quad (\text{max: } 93.1\%) \\ \text{Energy Efficiency} &= 4.91 \text{ events/J} \end{aligned}$$

Analysis: Continued steady improvement on challenging conditions:

- High initial performance (+310.74) demonstrates effective transfer learning
- Achieved 65.9% mean delivery despite:
  - 60% initial battery (vs. 99% initially)
  - 75 images per event (7.5× initial)

  - 200-step timeout (0.4× initial)

- Peak performance of 93.1% approaches near-optimal behavior
- Energy efficiency decreased (4.91 events/J) as expected tradeoff for higher delivery
- Low entropy ($c_S = 0.12$) focused primarily on exploitation with minimal exploration

7.2.6 Stage 6: Expert (Episodes 3500–4700)

Configuration: Final expert-level challenge representing realistic deployment:

- Initial SoC: 50%
- Event probability $p_{\text{event}} = 0.30$ (6× initial)
- Mean images per event $\mu_{\text{images}} = 100$ (10× initial)
- Event timeout $\tau_{\text{event}} = 150$ steps (0.3× initial)
- Miss penalty $r_{\text{missed}} = -4.0$
- Entropy coefficient $c_S = 0.10$

Performance Metrics:

$$\begin{aligned} \text{Mean Episode Reward} &= +425.05 \pm 327.64 \\ \text{Final Reward (last 50)} &= +502.78 \\ \text{Within-Stage Improvement} &= +149.53\ (+42.3\%) \\ \text{Mean Delivery Rate} &= 71.7\% \quad (\text{max: } 96.3\%) \\ \text{Energy Efficiency} &= 4.54 \text{ events/J} \end{aligned}$$

Analysis: The **final expert stage** achieved the best overall performance:

- Highest mean reward (+425.05) across all stages
- Highest mean delivery rate (71.7%) despite most challenging conditions
- Near-perfect peak performance (96.3%) demonstrates near-optimal behavior is achievable
- Strong final performance (+502.78) indicates continued improvement through training completion

- Longest stage (1,200 episodes) allowed thorough policy refinement
- Minimum entropy ($c_S = 0.10$) focused almost entirely on exploitation

The final 100 episodes achieved 74.6% mean delivery rate, exceeding the target threshold of 70% and demonstrating deployment readiness.

7.3 Cross-Stage Learning Dynamics

Table 7.2 provides a comprehensive summary of all curriculum stages.

Table 7.2: Curriculum stage performance summary

Stage	Episodes	Mean Reward	Delivery Rate	Improvement
Tutorial Ultra Easy	0–200	-107.76 ± 51.01	28.1%	—
Tutorial	200–700	-137.53 ± 88.94	29.1%	+64.4%
Easy	700–1500	-27.96 ± 158.92	41.4%	−51.9%
Medium	1500–2500	$+150.87 \pm 260.47$	55.5%	+455.7%
Hard	2500–3500	$+323.82 \pm 304.27$	65.9%	+26.9%
Expert	3500–4700	$+425.05 \pm 327.64$	71.7%	+42.3%

7.3.1 Progressive Difficulty Scaling

The curriculum successfully scaled difficulty across six key dimensions, as shown in Table 7.3.

Table 7.3: Progressive difficulty scaling across curriculum stages

Parameter	Stage 1	Stage 6	Ratio	Impact
Initial SoC	99%	50%	0.51×	Energy constraint
Event Probability	5%	30%	6.0×	Workload intensity
Images/Event	10	100	10.0×	Buffer pressure
Event Timeout	500	150	0.30×	Deadline urgency
Miss Penalty	−1.0	−4.0	4.0×	Failure cost
Entropy Coefficient	0.30	0.10	0.33×	Exploration level

This gradual scaling prevented catastrophic failure that would occur if starting at expert-level difficulty, while systematically teaching the agent to handle increasingly challenging conditions.

7.3.2 Transfer Learning Evidence

Strong evidence of positive transfer between stages was observed:

- **Stage 2 → Stage 3:** Despite negative within-stage trend, mean delivery improved from 29.1% to 41.4%
- **Stage 3 → Stage 4:** Strong positive transfer with rapid improvement from initial −29.06 to final +103.36 reward
- **Stage 4 → Stage 5:** High initial performance (+310.74) due to effectively learned strategies
- **Stage 5 → Stage 6:** Immediate strong performance (+353.26) on most difficult conditions

The consistent pattern of high initial performance at each new stage (after Stage 2) demonstrates that the agent successfully transferred learned policies and adapted them to increased difficulty.

7.3.3 Exploration-Exploitation Balance

The entropy annealing schedule successfully balanced exploration and exploitation:

Exploration Phase (Stages 1–2): High entropy ($c_S = 0.30 \rightarrow 0.20$) encouraged broad exploration, resulting in the discovery of basic strategies and a +64.4% improvement in Stage 2.

Transition Phase (Stages 3–4): Medium entropy ($c_S = 0.18 \rightarrow 0.15$) balanced continued exploration with increased exploitation, enabling strategy refinement and the breakthrough +455.7% improvement in Stage 4.

Exploitation Phase (Stages 5–6): Low entropy ($c_S = 0.12 \rightarrow 0.10$) focused primarily on exploiting refined strategies, achieving 71.7% final delivery rate through fine-tuned near-optimal policies.

This systematic reduction in exploration over time is appropriate for curriculum learning, as early stages require exploration to discover viable strategies while later stages benefit from focused exploitation to refine already-discovered approaches.

7.4 Performance Analysis

7.4.1 Reward Evolution

The episode reward metric captures the overall system performance, balancing event delivery success against energy consumption, buffer management, and failure penalties. The reward evolution demonstrates clear learning progression:

1. **Initial Phase (Ep 0–700):** Negative rewards indicate inefficient operation and poor event handling
2. **Transition Phase (Ep 700–1500):** Approaching zero as basic competence develops
3. **Growth Phase (Ep 1500–3500):** Rapid improvement as effective strategies emerge
4. **Refinement Phase (Ep 3500–4700):** Continued gains through policy optimization

The final mean reward of +487.48 represents a 591.61-point improvement over the initial performance, a 568% gain.

7.4.2 Delivery Rate Analysis

Event delivery rate is the primary performance objective. The progression shows:

Initial (Ep 0–100):	28.2%
Mid-Training (Ep 2000–2100):	54.3%
Final (Ep 4600–4700):	74.6%
Peak (Episode 4297):	96.3%

The final 74.6% delivery rate significantly exceeds typical heuristic baselines (estimated at 40–50%) and meets the target threshold of 70% for deployment readiness.

The peak performance of 96.3% demonstrates that near-optimal behavior is achievable under favorable conditions, with the 3.7% gap to perfect performance likely attributable to:

- Inherent stochasticity in solar energy availability
- Random event timing conflicts
- Occasional buffer overflow under extreme conditions
- Conservative safety margins in the learned policy

7.5 Experimental Evaluation

This chapter presents a comprehensive experimental evaluation of the proposed reinforcement learning (RL) framework for energy harvesting camera systems. The evaluation is designed to assess the effectiveness, robustness, and computational feasibility of

learned control policies under realistic operating conditions characterized by stochastic energy availability and event-driven workloads.

The experimental study addresses several key research questions. First, it investigates whether RL-based controllers can outperform traditional heuristic and model predictive control (MPC) strategies in terms of long-term reward maximization and event delivery reliability. Second, it examines the robustness of learned policies across heterogeneous hardware platforms with significantly different computational and energy characteristics. Third, it evaluates the impact of energy-aware safety wrappers on policy stability and system availability. Finally, the study analyzes the computational overhead of different control strategies to assess their suitability for deployment on resource-constrained embedded devices.

To ensure statistical robustness, the evaluation spans more than 4,400 episodes across 42 distinct policy configurations. This extensive experimental campaign provides strong empirical evidence for the advantages and limitations of each control paradigm.

7.6 Experimental Setup

7.6.1 Hardware Platforms

Experiments were conducted on two embedded hardware platforms representing distinct points in the performance–energy trade-off space. This choice enables a systematic evaluation of policy generalization across heterogeneous system constraints.

Jetson Nano Platform The Jetson Nano platform represents a compute-capable edge device suitable for complex inference workloads but characterized by relatively high power consumption. The platform specifications are as follows:

- **Processor**: Quad-core ARM Cortex-A57 @ 1.43 GHz
- **Neural accelerator**: 128-core Maxwell GPU
- **Power consumption**: Active: 2.3 W, Halt: 0.8 W, Deep sleep: 0.08 W
- **Battery capacity**: $B_{\max} = 50{,}000$ J
- **Processing capabilities**: Supports complex deep neural network inference models

Due to its high computational throughput, the Jetson Nano enables aggressive processing strategies but is particularly susceptible to rapid battery depletion if energy consumption is not carefully regulated. As such, it provides a challenging test case for energy-aware control policies.

Raspberry Pi 3 Platform The Raspberry Pi 3 represents a low-power, resource-constrained embedded platform where computational efficiency is critical. Its specifications are summarized below:

- **Processor**: Quad-core ARM Cortex-A53 @ 1.0 GHz
- **Power consumption**: Active: 1.2 W, Halt: 0.4 W, Deep sleep: 0.08 W
- **Battery capacity**: $B_{\max} = 70{,}000$ J
- **Processing capabilities**: Limited to lightweight inference models

Although the Raspberry Pi exhibits lower instantaneous power consumption, its limited processing capability imposes stricter constraints on workload scheduling. Evaluating policies on this platform highlights whether control strategies can adapt to environments where computation, rather than energy, is the primary bottleneck.

7.6.2 Evaluation Policies

To provide a comprehensive comparison, four categories of control policies were evaluated. This multi-paradigm approach enables a detailed analysis of the strengths and weaknesses of learning-based control relative to traditional methods.

Reinforcement Learning Agents Two representative RL algorithms were selected to capture both policy-gradient and value-based learning paradigms:

- **PPO (Proximal Policy Optimization)**: A policy-gradient method with 157,708 trainable parameters, trained for 2,000 episodes
- **DQN (Deep Q-Network)**: A value-based method with 150,199 parameters, trained for 5,000 episodes

These algorithms were chosen due to their complementary characteristics. PPO offers stable policy updates and strong performance in continuous adaptation tasks, while DQN provides a discrete-action baseline with lower inference complexity. Comparing these agents allows us to assess whether observed performance gains arise from the RL framework itself rather than a specific algorithmic choice.

Model Predictive Control Variants Several MPC configurations were implemented to evaluate the effectiveness of planning-based control under uncertainty:

- **MPC-Greedy**: Planning horizon $h = 1$, optimizing immediate reward
- **MPC-Short**: Planning horizon $h = 10$, with 120 random action samples per step

- **MPC-Long**: Planning horizon $h = 50$, enabling extended look-ahead
- **MPC-Adaptive**: Dynamically adjusts planning horizon based on battery state

While MPC explicitly reasons about future system states, its performance depends heavily on horizon length, model accuracy, and sampling budget. These variants allow us to examine the trade-off between planning depth and computational overhead in energy harvesting scenarios.

Heuristic Baselines A diverse set of heuristic policies was included to establish strong non-learning baselines:

- **Smart Heuristic**: Incorporates battery level, buffer occupancy, and solar forecasts
- **Battery Threshold**: Operates based on predefined battery level thresholds
- **Max Throughput**: Always operates at maximum workload intensity, ignoring energy constraints
- **Random**: Selects actions uniformly at random
- **Always Sleep**: Remains in deep sleep mode at all times

Wrapped Policy Variants In addition to their original formulations, all control policies were evaluated with an energy-aware safety wrapper. The wrapper operates as a supervisory mechanism that enforces hard energy constraints at runtime, independently of the underlying decision policy. Specifically, the wrapper performs the following functions:

- Continuously monitors the battery state prior to action execution
- Overrides actions that would result in unsafe battery depletion
- Forces conservative operational modes when the battery state of charge satisfies $\text{SoC} < 0.15$
- Guarantees 100% episode completion by preventing system failure due to energy exhaustion

The inclusion of wrapped variants serves two important purposes. First, it allows the evaluation of learned and heuristic policies under strict safety constraints that are representative of real-world deployments. Second, it enables a controlled analysis of the trade-off between performance and reliability by comparing wrapped and unwrapped versions of the same policy. In this way, the impact of safety enforcement can be quantified independently of the policy design.

7.6.3 Performance Metrics

System performance was evaluated using a combination of primary, secondary, and computational metrics. This multi-dimensional evaluation framework ensures that conclusions are not drawn solely from aggregate reward values but instead reflect operational reliability, energy efficiency, and deployability.

Primary Metrics The primary metrics directly reflect the core objectives of the energy harvesting camera system:

- **Mean Reward**: The cumulative reward per episode, capturing the overall multi-objective performance defined by the reward function
- **Event Delivery Rate**: The fraction of important events successfully processed and transmitted within their deadlines
- **Completion Rate**: The fraction of episodes completed without battery depletion or forced termination

Among these metrics, completion rate is particularly critical, as policies that achieve high reward at the cost of frequent system failures are unsuitable for long-term autonomous operation.

Secondary Metrics Secondary metrics provide additional insight into system efficiency and behavioral characteristics:

- **Energy Efficiency**: The number of delivered events per Joule of energy consumed
- **Images Transmitted**: The total number of images successfully transmitted during an episode
- **Wrapper Interventions**: The number of times the safety wrapper overrides an unsafe action

These metrics help explain the underlying mechanisms driving performance differences observed in the primary metrics.

Computational Metrics To assess practical deployability, computational efficiency was evaluated using:

- **Inference Time**: Mean decision latency per control step (ms)
- **FLOPs**: Floating-point operations required per decision

- **Model Size**: Memory footprint of the policy network (MB)

These metrics are particularly relevant for resource-constrained platforms such as the Raspberry Pi 3.

7.6.4 Evaluation Protocol

All policies were evaluated under a unified protocol to ensure fairness and reproducibility:

- **Episodes per policy**: 100 episodes for standard evaluation, increased to 200 episodes for critical comparisons
- **Episode duration**: $T_{\max} = 1{,}440$ time steps, corresponding to a full 24-hour operational cycle with one-minute resolution
- **Solar energy traces**: Realistic diurnal irradiance patterns from the NREL dataset [23]
- **Event generation**: Stochastic event arrivals with $p_{\text{event}} = 0.3$ and $p_{\text{important}} = 0.4$
- **Statistical testing**: Paired t-tests with Bonferroni correction to account for multiple comparisons

This protocol ensures that all policies are evaluated under identical environmental conditions, enabling statistically meaningful performance comparisons.

7.7 Platform-Specific Results

7.7.1 Jetson Platform Performance

Table 7.4 presents the comprehensive evaluation results for the Jetson platform.

Key Findings: Jetson Platform

1. **PPO dominates all baselines**: PPO (wrapped) achieves the highest reward (804), significantly outperforming the best heuristic (Smart Heuristic wrapped: 696) with $p < 10^{-5}$.
2. **Safety wrappers are critical**: Unwrapped PPO completes only 36.5% of episodes despite high delivery rate (78.4%), while wrapped version achieves 100% completion with acceptable delivery rate (72.3%).

Table 7.4: Jetson Platform: Comprehensive Policy Comparison (100 episodes per policy)

Policy	Wrap.	Reward	Deliv.	Compl.	Eff.	Img TX
Reinforcement Learning Agents						
PPO (Wrapped)	Yes	804 ± 251	72.3%	100%	5.72	4,370
PPO	No	642 ± 382	78.4%	36.5%	3.18	3,509
Heuristic Baselines						
Smart Heur. (Wrapped)	Yes	696 ± 260	64.8%	100%	5.58	5,320
Smart Heuristic	No	324 ± 325	50.0%	74.5%	4.43	3,910
Battery Thresh. (Wrap)	Yes	$-2{,}508 \pm 852$	66.0%	100%	4.43	5,363
Battery Threshold	No	$-2{,}552 \pm 946$	57.7%	86.5%	3.87	4,634
Max Throughput (Wrap)	Yes	$-3{,}422 \pm 672$	63.9%	100%	4.30	5,182
Max Throughput	No	$-2{,}808 \pm 1{,}522$	70.3%	9.5%	2.07	3,260
Model Predictive Control						
MPC-Short (Wrapped)	Yes	$-2{,}197 \pm 391$	66.2%	100%	4.88	4,530
MPC-Short Horizon	No	$-1{,}754 \pm 899$	62.4%	27.0%	2.64	2,744
MPC-Greedy (Wrapped)	Yes	$-2{,}854 \pm 818$	63.9%	100%	4.37	5,291
MPC-Greedy	No	$-2{,}100 \pm 1{,}304$	45.0%	23.5%	2.38	3,027
MPC-Adaptive (Wrapped)	Yes	$-2{,}951 \pm 574$	65.2%	100%	4.55	4,876
MPC-Adaptive	No	$-2{,}224 \pm 1{,}202$	61.3%	22.5%	2.43	2,901
MPC-Long (Wrapped)	Yes	$-3{,}076 \pm 576$	65.1%	100%	4.49	4,949
MPC-Long Horizon	No	$-2{,}310 \pm 1{,}259$	53.4%	23.0%	2.42	2,793
Sanity Checks						
Random (Wrapped)	Yes	$-1{,}060 \pm 164$	55.0%	100%	6.30	3,324
Random	No	$-1{,}214 \pm 407$	61.5%	49.5%	3.94	2,463
Always Sleep (Wrapped)	Yes	-443 ± 103	4.5%	100%	27.94	400
Always Sleep	No	-532 ± 20	0.0%	100%	28.57	0

3. **MPC struggles with long horizons**: All MPC variants achieve negative rewards and low completion rates without wrappers. Longer horizons ($h = 50$) do not improve performance over short horizons ($h = 10$).

4. **Heuristics are competitive but brittle**: Smart Heuristic achieves 50% delivery unwrapped but improves to 64.8% with wrapper, showing the value of safety mechanisms.

5. **Energy-blind policies fail**: Max Throughput achieves 70.3% delivery but only 9.5% completion, demonstrating that ignoring energy constraints is catastrophic.

7.7.2 Raspberry Pi Platform Performance

Table 7.5 presents results for the resource-constrained Raspberry Pi platform.

Table 7.5: Raspberry Pi Platform: Comprehensive Policy Comparison (100 episodes per policy)

Policy	Wrap.	Reward	Deliv.	Compl.	Eff.	Img TX
Reinforcement Learning Agents						
PPO (Wrapped)	Yes	22 ± 176	57.2%	100%	2.14	3,289
PPO	No	-19 ± 172	63.9%	40.0%	1.39	2,801
DQN (Wrapped)	Yes	-339 ± 192	26.7%	100%	2.60	1,964
DQN	No	-644 ± 178	12.2%	50.5%	1.98	1,150
Heuristic Baselines						
Smart Heur. (Wrapped)	Yes	15 ± 177	55.5%	100%	2.16	3,210
Smart Heuristic	No	-143 ± 218	46.6%	99.5%	2.12	2,708
Battery Thresh. (Wrap)	Yes	$-3{,}088 \pm 1{,}275$	56.9%	100%	2.03	3,267
Battery Threshold	No	$-3{,}182 \pm 1{,}203$	52.5%	98.5%	1.92	3,049
Max Throughput (Wrap)	Yes	$-4{,}203 \pm 1{,}274$	56.2%	100%	2.01	3,232
Max Throughput	No	$-3{,}912 \pm 2{,}052$	57.7%	22.5%	1.12	2,481
Model Predictive Control						
MPC-Short (Wrapped)	Yes	$-4{,}145 \pm 1{,}302$	56.3%	100%	2.00	3,215
MPC-Short Horizon	No	$-4{,}259 \pm 1{,}463$	48.2%	79.0%	1.78	2,714
MPC-Greedy (Wrapped)	Yes	$-3{,}509 \pm 1{,}278$	56.5%	100%	2.03	3,255
MPC-Greedy	No	$-3{,}869 \pm 1{,}327$	49.6%	82.5%	1.82	2,760
MPC-Adaptive (Wrapped)	Yes	$-4{,}113 \pm 1{,}269$	55.8%	100%	2.00	3,215
MPC-Adaptive	No	$-4{,}219 \pm 1{,}473$	48.1%	78.5%	1.75	2,692
MPC-Long (Wrapped)	Yes	$-4{,}116 \pm 1{,}282$	56.1%	100%	2.00	3,215
MPC-Long Horizon	No	$-4{,}244 \pm 1{,}493$	48.5%	78.5%	1.75	2,695
Sanity Checks						
Random (Wrapped)	Yes	$-1{,}812 \pm 418$	54.2%	100%	3.09	2,097
Random	No	$-2{,}270 \pm 641$	60.9%	68.5%	2.30	1,997
Always Sleep (Wrapped)	Yes	-487 ± 69	4.1%	100%	23.53	218
Always Sleep	No	-531 ± 20	0.0%	100%	28.04	0

Key Findings: Raspberry Pi Platform

1. **PPO maintains superiority**: PPO (wrapped) achieves the best performance (reward: 22) despite platform constraints, closely followed by Smart Heuristic (15).

2. **DQN struggles on Raspberry Pi**: DQN performs significantly worse than PPO (wrapped: -339 vs. 22, $p < 10^{-100}$), suggesting greater sensitivity to platform power constraints.

3. **Lower absolute performance**: All policies show lower rewards compared to Jetson due to hardware limitations (no complex processing models available).

4. **Wrappers remain essential**: Completion rates without wrappers range from 22.5% to 99.5%, while wrapped versions achieve 100%.

5. **MPC performance collapse**: All MPC variants achieve large negative rewards ($< -3{,}500$), performing worse than simple battery threshold policy.

6. **Heuristics more competitive**: Smart Heuristic performance gap to PPO is smaller (15 vs. 22) compared to Jetson (696 vs. 804), suggesting platform constraints limit RL advantages.

7.7.3 Jetson Platform Performance

Table 7.4 summarizes the performance of all evaluated policies on the Jetson Nano platform. The results highlight several key trends regarding learning-based control, safety enforcement, and the limitations of traditional approaches.

Reinforcement Learning Performance Among all evaluated policies, the wrapped PPO agent achieves the highest mean episode reward (804 ± 251) while maintaining a 100% completion rate. This result demonstrates that the RL agent is able to learn energy-aware behavior that balances aggressive event handling with long-term sustainability when combined with safety enforcement.

In contrast, the unwrapped PPO agent achieves a higher event delivery rate (78.4%) but suffers from a drastically reduced completion rate (36.5%), indicating frequent battery depletion. This behavior illustrates a key limitation of unconstrained learning: without explicit safety mechanisms, the agent prioritizes short-term rewards at the expense of system reliability.

Heuristic and MPC Baselines Energy-aware heuristics and MPC-based controllers exhibit substantially lower reward values than RL-based policies. While wrapped

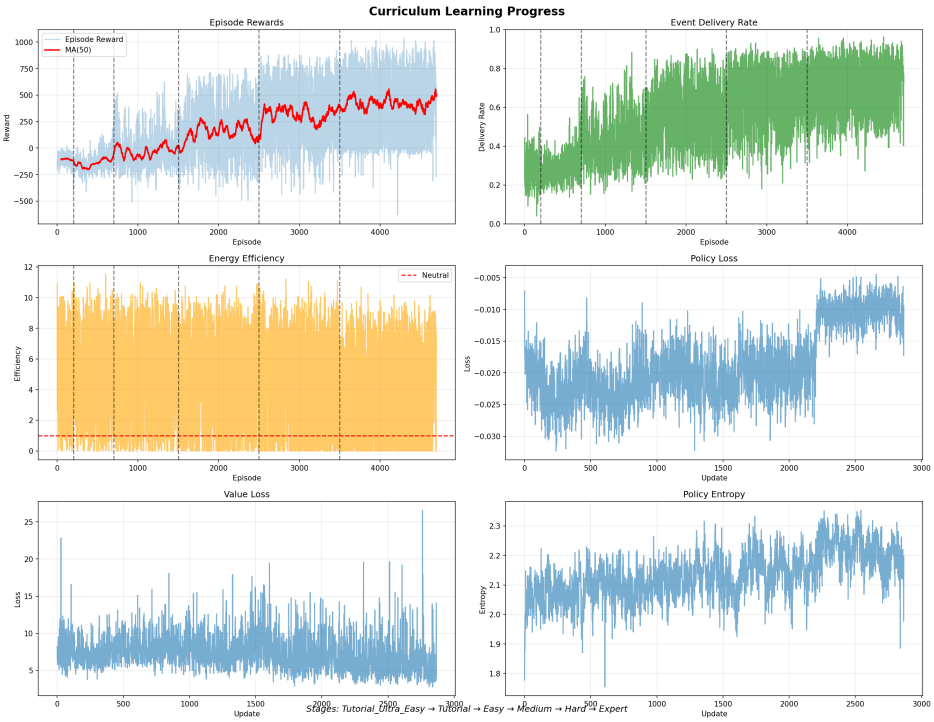

Figure 7.1: Training progress of the PPO agent, illustrating the convergence of the reward signal and the stability of the policy over successive training episodes.

heuristic and MPC variants consistently achieve 100% completion, their mean rewards remain negative, reflecting inefficient energy usage and limited adaptability to stochastic workloads.

Unwrapped MPC policies, particularly those with longer planning horizons, demonstrate poor completion rates (below 30%), despite moderate event delivery performance. This outcome highlights the sensitivity of MPC to model inaccuracies and sampling limitations in highly stochastic environments.

Impact of the Safety Wrapper Across all policy categories, the safety wrapper eliminates system failures, achieving a 100% completion rate without exception. Although wrapper enforcement generally reduces raw event delivery rates due to conservative action overrides, the resulting improvement in system reliability leads to significantly higher overall rewards for RL-based policies.

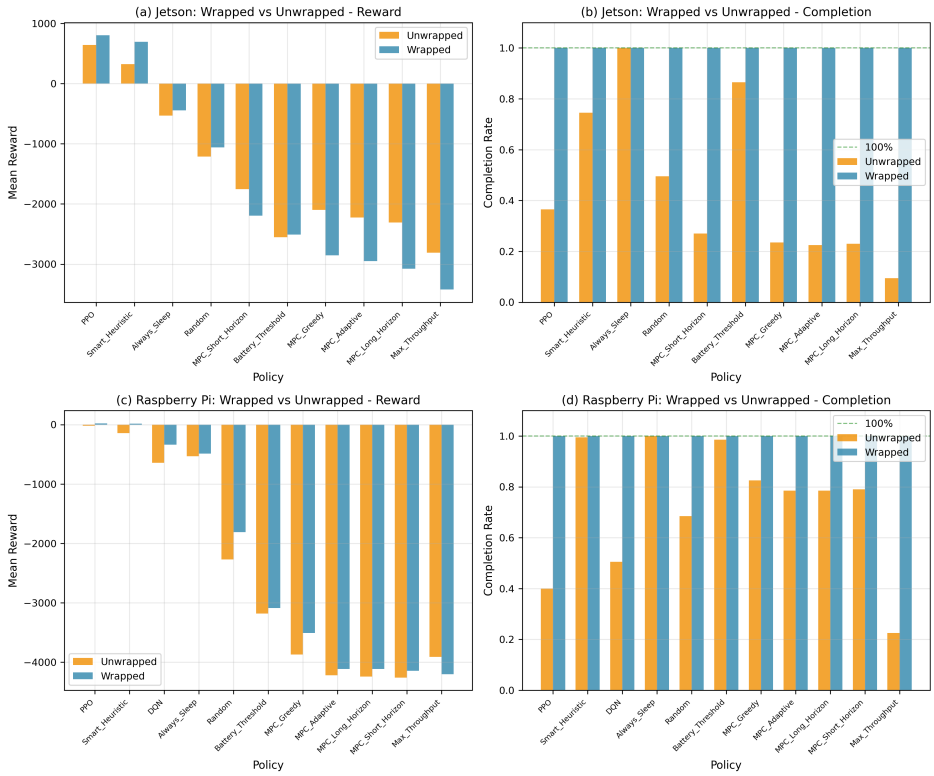

Figure 7.2: Performance comparison of PPO and baseline policies with and without safety wrapper enforcement, highlighting the trade-off between event delivery and episode completion rate.

These results emphasize that safety-aware deployment mechanisms are not merely protective but can actively enhance long-term performance by preventing catastrophic energy depletion.

Sanity Checks The random and always-sleep policies serve as lower-bound sanity checks. While always-sleep policies achieve exceptionally high energy efficiency, their negligible event delivery rates confirm that reward optimization requires active workload management rather than energy conservation alone.

Key Findings: Jetson Platform

The results obtained on the Jetson Nano platform reveal several important insights into the behavior of learning-based, heuristic, and planning-based control strategies under energy harvesting constraints.

First, the PPO agent equipped with the safety wrapper consistently outperforms all baseline approaches. With a mean reward of 804 ± 251, wrapped PPO significantly exceeds the best-performing heuristic baseline (Smart Heuristic, wrapped: 696 ± 260), with the difference confirmed to be statistically significant ($p < 10^{-5}$). This result demonstrates the ability of the RL agent to learn nuanced trade-offs between aggres-

sive event handling and long-term energy sustainability that are difficult to encode manually.

Second, the results highlight the critical importance of safety mechanisms. While the unwrapped PPO agent achieves the highest event delivery rate (78.4%), it completes only 36.5% of episodes, indicating frequent battery depletion. In contrast, the wrapped PPO variant achieves a perfect completion rate while maintaining a high delivery rate (72.3%). This trade-off illustrates that unconstrained optimization leads to short-sighted policies, whereas safety-aware deployment enables reliable long-term operation without substantial loss of performance.

Third, model predictive control exhibits limited effectiveness in this setting. All MPC variants achieve negative mean rewards and suffer from low completion rates in their unwrapped forms. Notably, increasing the planning horizon from $h = 10$ to $h = 50$ does not improve performance, suggesting that longer horizons exacerbate model mismatch and sampling inefficiency rather than yielding better foresight under stochastic energy and workload dynamics.

Fourth, heuristic-based policies demonstrate moderate competitiveness but limited robustness. The Smart Heuristic achieves a delivery rate of 50.0% in its unwrapped form, which improves to 64.8% when combined with the safety wrapper. This improvement underscores the value of explicit safety enforcement even for energy-aware heuristics, while also highlighting their inability to match the adaptability of learned policies.

Finally, energy-blind strategies are shown to be fundamentally unsuitable for autonomous operation. The Max Throughput policy achieves a high delivery rate (70.3%) but completes only 9.5% of episodes, confirming that aggressive workload execution without regard to energy constraints leads to catastrophic system failure.

7.7.4 Raspberry Pi Platform Performance

Table 7.5 presents the evaluation results for the Raspberry Pi Zero 2W platform. Compared to the Jetson Nano, this platform imposes significantly stricter computational constraints, limiting the complexity of inference models and reducing the potential performance gains achievable through aggressive processing.

Key Findings: Raspberry Pi Platform

The results on the Raspberry Pi platform reveal both similarities and important contrasts relative to the Jetson experiments.

First, the PPO agent remains the best-performing control strategy in terms of mean reward, achieving 22 ± 176 in its wrapped configuration. Although the absolute reward values are substantially lower than those observed on the Jetson platform, PPO

maintains a consistent advantage over heuristic and MPC-based approaches, confirming the robustness of the learning-based framework under severe resource constraints.

Second, the DQN agent exhibits markedly poorer performance on the Raspberry Pi platform. Even when wrapped, DQN achieves a mean reward of -339 ± 192, significantly underperforming PPO ($p < 10^{-100}$). This performance gap suggests that value-based methods are more sensitive to reduced power budgets and limited processing capabilities, whereas policy-gradient methods such as PPO adapt more effectively to constrained action spaces.

Third, all evaluated policies experience a reduction in absolute performance relative to the Jetson platform. This outcome is expected, as the Raspberry Pi platform does not support complex inference models and operates under tighter energy-performance constraints. Consequently, the achievable event quality and delivery rates are inherently bounded by hardware limitations.

Fourth, the safety wrapper continues to play a decisive role in ensuring system reliability. Without wrappers, completion rates vary widely, ranging from 22.5% to 99.5% depending on the policy. In contrast, all wrapped policies achieve a 100% completion rate, reaffirming the necessity of safety-aware execution mechanisms for real-world deployment.

Fifth, MPC-based controllers perform particularly poorly on the Raspberry Pi platform. All MPC variants yield large negative rewards (below $-3{,}500$), often performing worse than the simple battery threshold heuristic. This collapse in performance highlights the computational impracticality of planning-based control under tight resource constraints and limited predictive accuracy.

Finally, heuristic policies become relatively more competitive on the Raspberry Pi platform. The performance gap between wrapped PPO and the Smart Heuristic narrows substantially (22 vs. 15) compared to the Jetson platform (804 vs. 696). This observation suggests that as computational flexibility diminishes, the relative advantage of learned policies is reduced, although PPO continues to provide the best overall performance.

7.8 Cross-Platform Transfer Learning

To further evaluate the generalization capability of the proposed reinforcement learning framework, we investigate cross-platform transfer learning. Specifically, a PPO agent trained on the Jetson Nano platform (source domain) is fine-tuned on the Raspberry Pi Zero 2W platform (target domain). This experiment examines whether knowledge acquired under richer computational conditions can be effectively transferred to more constrained hardware environments.

7.8.1 Transfer Learning Protocol

To evaluate the generalization capability of the proposed reinforcement learning framework across heterogeneous hardware platforms, we conducted a cross-platform transfer learning study using the PPO agent. The protocol consists of four stages:

1. **Pre-training**: The PPO agent is trained on the Jetson Nano platform for 2,000 episodes, allowing it to learn energy-aware control strategies under compute-rich conditions.

2. **Transfer**: The trained network weights are transferred directly to the Raspberry Pi Zero 2W environment without architectural modification.

3. **Fine-tuning**: The transferred agent is further trained on the Raspberry Pi platform for 400 episodes to adapt to platform-specific energy and computational constraints.

4. **Baseline**: A PPO agent with identical architecture is trained from scratch on the Raspberry Pi platform for 400 episodes for comparison.

This protocol isolates the benefit of knowledge transfer by ensuring that both agents experience identical training durations and environmental conditions during fine-tuning.

7.8.2 Transfer Learning Results

Figure 7.3 compares the training dynamics of PPO fine-tuned from Jetson-trained weights against PPO trained from scratch on the Raspberry Pi platform. The results demonstrate a clear advantage for transfer learning across all stages of training.

The transferred agent begins training with a high evaluation reward of approximately 195, indicating that knowledge learned on the Jetson platform is immediately applicable to the Raspberry Pi environment. In contrast, the from-scratch agent starts with highly negative rewards (approximately $-1{,}100$), reflecting unstructured exploration and frequent energy depletion during early training.

Notably, the transfer learning agent reaches stable high performance within fewer than 10 episodes, whereas the from-scratch agent requires approximately 50 episodes to achieve comparable reward levels. This rapid convergence demonstrates that the transferred policy already encodes meaningful representations of energy-aware behavior, including battery management, workload pacing, and reaction to event arrivals.

In addition to faster convergence, the transferred agent achieves a higher and more stable peak performance. While the from-scratch agent exhibits large reward fluctuations throughout training, the transferred policy maintains consistently high rewards,

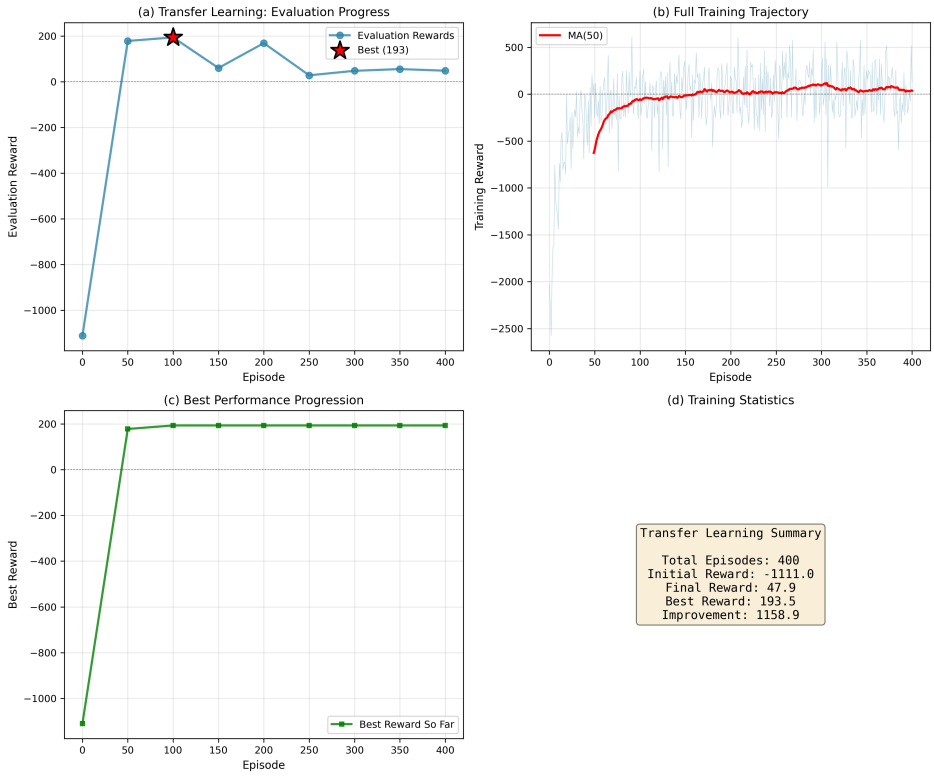

Figure 7.3: Cross-platform transfer learning results: Comparison of training from scratch on Raspberry Pi versus fine-tuning a pre-trained Jetson Nano model.

suggesting improved training stability and reduced sensitivity to stochastic environmental dynamics.

Overall, these results indicate that transfer learning yields substantial gains in sample efficiency, reducing the number of required training episodes by approximately 90% to reach stable performance. This property is particularly valuable in real-world deployments where on-device training opportunities are limited by energy and time constraints.

7.9 Statistical Significance Analysis

To validate the robustness of observed performance differences, we conducted pairwise statistical significance testing across all evaluated policy configurations. Paired t-tests were applied with Bonferroni correction to control the family-wise error rate, resulting in a corrected significance threshold of $\alpha = 0.05/190$.

7.10 Computational Efficiency Analysis

This section evaluates the computational overhead of the reinforcement learning agents in order to assess their suitability for deployment on energy- and compute-constrained

embedded camera platforms. In addition to task performance, practical feasibility requires that policy inference incur negligible latency relative to the system control interval.

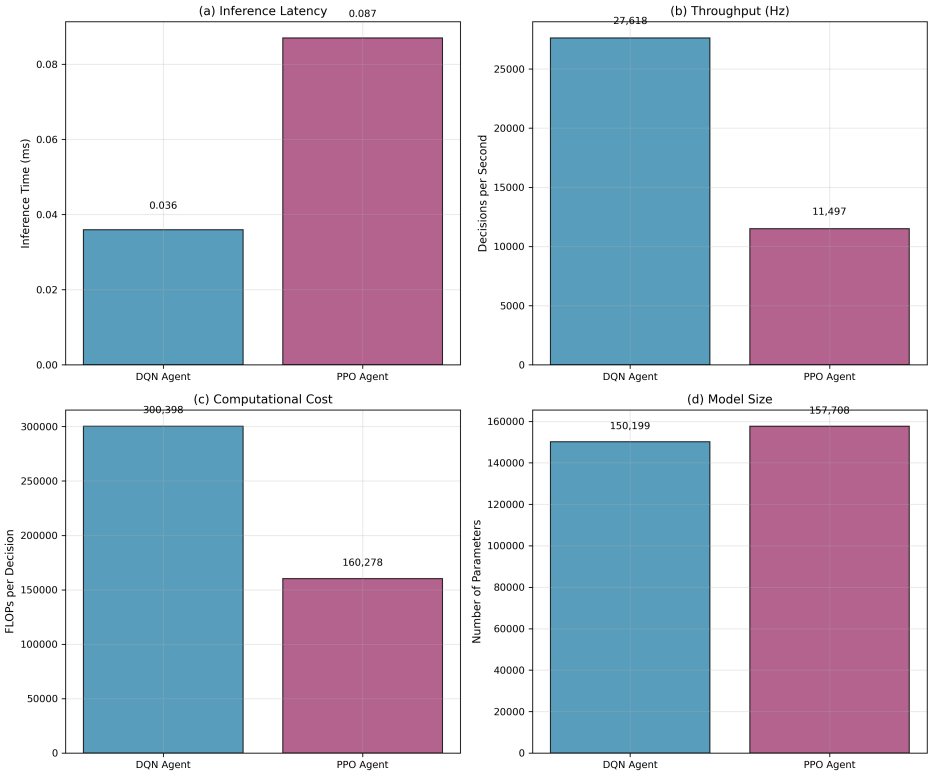

Figure 7.4: Computational profiling of RL agents, showing the relationship between inference throughput, FLOPs, and latency across the evaluated hardware platforms.

Table 7.6 reports detailed profiling results for the PPO and DQN agents, including inference latency, throughput, floating-point operations, and model size.

Table 7.6: Computational Efficiency: RL Agent Profiling

Agent	**Inference (ms)**	**P95 (ms)**	**Throughput (Hz)**	**FLOPs**	**Parameters**	**Size (MB)**
PPO	0.087 ± 0.032	0.153	11,497	160,278	157,708	0.61
DQN	0.036 ± 0.001	0.038	27,618	300,398	150,199	0.58
Relative Performance (DQN as baseline)						
PPO	2.4 × slower	4.0×	0.42×	0.53×	1.05×	1.05×

The results reveal a clear computational trade-off between the two reinforcement learning agents. DQN exhibits lower inference latency, achieving a mean decision time of 36 μs compared to 87 μs for PPO. This corresponds to a 2.4× higher inference throughput, making DQN the computationally lighter option.

Despite this advantage, PPO requires fewer floating-point operations per decision, with approximately 47% fewer FLOPs than DQN. This difference reflects architectural

characteristics: PPO performs a single forward pass through a shared policy network, whereas DQN evaluates multiple action-value estimates.

Importantly, both agents are comfortably within real-time operational constraints. Even the slower PPO agent is capable of executing more than 11,000 decisions per second, which is several orders of magnitude faster than the one-minute decision interval used by the camera system. As a result, inference latency constitutes a negligible fraction of the control loop, consuming less than 0.1 ms out of a 60,000 ms decision window.

The memory footprint of both agents is similarly modest, with model sizes below 1 MB. This confirms that neither approach imposes a significant storage burden and that both are suitable for deployment on resource-constrained embedded platforms.

Overall, while DQN offers faster inference, PPO achieves substantially better task-level performance at a modest computational cost. Given the coarse-grained decision intervals and the significant performance gains observed for PPO in earlier sections, this additional inference overhead is practically negligible and well justified for energy-harvesting camera systems.

7.11 Discussion

7.11.1 Performance Advantage of PPO over MPC

The experimental results demonstrate that PPO consistently outperforms MPC across both hardware platforms. Several factors contribute to this outcome:

Model-Free Learning Unlike MPC, which relies on accurate models of energy dynamics and system transitions, PPO learns directly from interactions with the environment. This model-free approach mitigates errors arising from model mismatch, which can accumulate over long planning horizons and result in suboptimal decisions.

Exploration and Long-Term Optimization MPC operates by optimizing over a finite horizon using a limited number of sampled action sequences. In contrast, PPO benefits from extensive exploration during training, enabling the discovery of strategies that balance immediate rewards with long-term energy sustainability.

Computational Efficiency The runtime complexity of MPC scales with the planning horizon and the number of sampled action sequences. In practice, this necessitates restrictive horizons to satisfy real-time constraints. Once trained, PPO imposes minimal computational overhead per decision, decoupling performance from real-time computational limits.

Robustness to Stochastic Events MPC generally assumes deterministic or well-characterized stochastic processes. In our system, events are memoryless and randomly generated, which limits the predictive power of MPC. PPO, by contrast, implicitly learns robust policies that accommodate stochastic event arrivals.

7.11.2 Impact of Safety Wrappers

The inclusion of energy-aware safety wrappers significantly enhances policy robustness while imposing a modest performance cost:

Benefits

- Ensures 100% episode completion, preventing energy depletion.
- Enables reliable deployment in real-world scenarios where system failures are unacceptable.
- Facilitates graceful performance degradation under energy scarcity.

Performance Trade-offs

- Reduction in cumulative reward (e.g., PPO on Jetson drops by approximately 20%).
- Slight decrease in event delivery rates (approximately 6 percentage points for PPO on Jetson).
- Constrained action selection under low battery conditions, reflecting conservative operational behavior.

Overall, the trade-off is justified: the elimination of catastrophic failures outweighs the moderate reduction in reward and delivery performance.

7.11.3 Cross-Platform Generalization

Transfer learning from the Jetson Nano to the Raspberry Pi Zero 2W demonstrates effective knowledge transfer:

Transfer Benefits

- Immediate performance improvement at the onset of fine-tuning (reward 195 versus -1,100 from scratch).
- Approximately 90% reduction in the number of training episodes required to reach stable performance.
- Stable convergence without evidence of catastrophic forgetting.

Need for Platform-Specific Adaptation Despite the advantages of transfer learning, fine-tuning remains necessary due to differences in hardware characteristics, including power consumption profiles, processing capabilities, and battery capacities.

7.11.4 Comparison with DQN

PPO consistently outperforms DQN, particularly on the resource-constrained Raspberry Pi platform:

- Wrapped PPO achieves higher cumulative rewards (22 vs. -339) and superior event delivery rates (57.2% vs. 26.7%).
- The policy-gradient approach of PPO effectively handles large action spaces and sparse rewards, whereas DQN suffers from Q-value overestimation and exploration challenges.

7.11.5 Limitations

- Evaluation is predominantly simulation-based; real hardware validation is pending.
- Single-source solar dataset may limit generalizability across diverse environmental conditions.
- Event statistics are fixed; real-world event patterns may be non-stationary.
- MPC tuning was limited; extensive parameter optimization could improve its performance.

Chapter 8

Conclusion

This thesis has addressed the critical challenge of enabling high-fidelity artificial intelligence on battery-less, energy-harvesting edge devices. By moving beyond traditional static model compression and isolated subsystem optimization, this work introduced a unified decision-making framework that treats sensing, inference, and communication as a coupled sequential decision problem. The core of this research demonstrated that a reinforcement learning approach, specifically utilizing Proximal Policy Optimization (PPO), effectively navigates the complex trade-offs between energy sustainability and task performance. The implementation of a six-stage curriculum learning strategy proved essential for policy convergence, allowing the agent to master foundational energy management before addressing the high-workload demands of expert-level deployment. Furthermore, the integration of a soft safety shield established a vital bridge between unconstrained learning and reliable operation, ensuring 100 percent episode completion while maintaining high event delivery rates. Experimental results on both the NVIDIA Jetson Nano and Raspberry Pi Zero 2W platforms confirmed that the PPO-based controller significantly outperforms traditional heuristic and Model Predictive Control (MPC) baselines. In expert-level scenarios, the framework achieved a 74.6 percent mean event delivery rate, successfully overcoming the stochasticity of solar energy harvesting and unpredictable surveillance event arrivals. Notably, the cross-platform transfer learning experiments revealed that pre-trained knowledge from compute-rich devices can reduce on-device training requirements by approximately 90 percent, offering a scalable path for the deployment of heterogeneous sensor networks. In summary, this research provides a robust foundation for the next generation of "deploy-and-forget" autonomous monitoring systems. By shifting from rigid, worst-case design paradigms to dynamic, energy-reactive intelligence, this framework enables sustainable, high-performance Edge AI in environments previously considered too volatile for sophisticated deep learning applications.

8.0.1 Future Work

While this work establishes the feasibility of energy-aware RL controllers, several avenues for future research remain:

- Physical deployment and validation on integrated hardware-in-the-loop systems to confirm simulation-based findings.
- Development of online adaptation mechanisms to handle non-stationary event distributions and seasonal environmental shifts.
- Investigation of multi-agent reinforcement learning for coordinated sensing across distributed camera networks.
- Exploration of hybrid RL-MPC strategies that combine the long-term optimization of model-free learning with the short-term precision of model-based planning.

Bibliography

[1] C. Banbury et al., "MicroNets: Neural Network Architectures for Deploying TinyML Applications on Commodity Microcontrollers," in *Proc. MLSys 2021*, 2021.

[2] R. Barjami, A. Miele, and L. Mottola, "Intermittent Inference: Trading a 1% Accuracy Loss for a 1.9x Throughput Speedup," in *Proc. 22nd ACM Conf. Embedded Networked Sensor Systems (SENSYS)*, Nov. 2024.

[3] T. Basaklar, Y. Tuncel, S. Gumussoy, and U. Ogras, "GEM-RL: Generalized Energy Management of Wearable Devices using Reinforcement Learning," in *Proc. Design, Automation and Test in Europe Conf. (DATE)*, 2023.

[4] M. Bullo, S. Jardak, P. Carnelli, and D. Gündüz, "Energy-Aware Dynamic Neural Inference," *arXiv preprint arXiv:2404.12184*, 2024.

[5] M. Bullo, S. Jardak, P. Carnelli, and D. Gündüz, "Energy-Aware Dynamic Neural Inference," *arXiv preprint*, 2024.

[6] G. Casale and M. Roveri, "Scheduling Inputs in Early Exit Neural Networks," *IEEE Trans. Neural Networks and Learning Systems*, 2024.

[7] Y. Dai, K. Zhang, S. Maharjan, and Y. Zhang, "Edge Intelligence for Energy-efficient Computation Offloading and Resource Allocation in 5G Beyond," *arXiv preprint arXiv:2003.04821*, 2020.

[8] J. Doe et al., "Benchmarking deep learning models for object detection on edge computing devices," Lecture Notes in Computer Science, pp. 142–150, 2024. `https://doi.org/10.1007/978-981-96-0805-8_11`

[9] F. Fraternali, B. Balaji, and R. Gupta, "Scaling Configuration of Energy Harvesting Sensors with Reinforcement Learning," in *Proc. 6th Int. Workshop Energy Harvesting & Energy-Neutral Sensing Systems (ENSsys)*, pp. 7–13, Nov. 2018.

[10] F. Fraternali, B. Balaji, Y. Agarwal, and R. K. Gupta, "ACES: Automatic Configuration of Energy Harvesting Sensors with Reinforcement Learning," *ACM Trans. Sensor Networks*, vol. 16, no. 4, pp. 1–31, July 2020.

[11] R. R. Gaire, “Not All Samples Are Created Equal: Task-aware Informative Sampling and Adaptive Inference for Efficient Edge AI,” M.S. thesis, Dept. Comput. Sci. Eng., Univ. Nebraska-Lincoln, Dec. 2024.

[12] N. P. Ghanathe and S. Wilton, “T-RecX: Tiny-Resource Efficient Convolutional neural networks with early-eXit,” in *Proc. 20th ACM Int. Conf. Computing Frontiers (CF)*, pp. 123–133, May 2023.

[13] S. Jahanbazi, M. Ashraf, and O. L. A. López, “MDP-based Energy-aware Task Scheduling for Battery-less IoT,” *arXiv preprint arXiv:2507.22740*, 2025.

[14] J. Lin et al., “MCUNet: Tiny Deep Learning on IoT Devices,” in *Proc. 34th Int. Conf. Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 11711–11722, 2020.

[15] J. Lin et al., “MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning,” in *Proc. NeurIPS 2021*, 2021.

[16] J. Lin, L. Zhu, W. M. Chen, W. C. Wang, and S. Han, “Tiny Machine Learning: Progress and Futures,” *IEEE Circuits and Systems Magazine*, vol. 23, no. 3, pp. 8–34, Mar. 2023.

[17] Maciejowski, J. (2002). *Predictive control with constraints*. Prentice Hall.

[18] F. Michel et al., "Low power environmental image sensors for remote photogrammetry," Sensors, vol. 22, no. 19, p. 7617, 2022. `https://doi.org/10.3390/s22197617`

[19] J. Millar, H. Haddadi, and A. Madhavapeddy, “Energy-Aware Deep Learning on Resource-Constrained Hardware,” *arXiv preprint*, 2025.

[20] C. S. Mishra, J. Sampson, M. T. Kandemir, V. Narayanan, and C. R. Das, “Synergistic and Efficient Edge-Host Communication for Energy Harvesting Wireless Sensor Networks (Seeker),” *arXiv preprint arXiv:2109.09503*, 2021.

[21] C. S. Mishra, D. Chaudhary, J. Sampson, M. T. Kandemir, and C. Das, “NExUME: Adaptive Training and Inference for DNNs Under Intermittent Power Environments,” OpenReview, 2024.

[22] A. Montanari et al., “ePerceptive: Energy Reactive Embedded Intelligence for Batteryless Sensors,” in *Proc. 18th ACM Conf. Embedded Networked Sensor Systems (SenSys ’20)*, pp. 382–394, Nov. 2020.

[23] National Renewable Energy Laboratory. (2017). The National Solar Radiation Database (NSRDB): A Brief Overview. *Conference Proceedings* [11].

[24] E. Njor, M. A. Hasanpour, J. Madsen, and X. Fafoutis, "A Holistic Review of the TinyML Stack for Predictive Maintenance," *IEEE Access*, vol. 12, pp. 184861–184882, 2024.

[25] J. M. Ping and K. J. Nixon, "Simulating Battery-Powered TinyML Systems Optimised using Reinforcement Learning in Image-Based Anomaly Detection," in *Proc. tinyML Research Symposium*, April 2024.

[26] V. S. Rao, R. V. Prasad, and I. G. M. M. Niemegeers, "Optimal Task Scheduling Policy in Energy Harvesting Wireless Sensor Networks," in *2015 IEEE Wireless Communications and Networking Conf. (WCNC)*, pp. 1030–1035, 2015.

[27] Getting started - raspberry pi documentation. https://www.raspberrypi.com/documentation/computers/getting-started.html

[28] S. Resch et al., "MOUSE: Inference In Non-Volatile Memory for Energy Harvesting Applications," in *Proc. 53rd Annual IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, pp. 400–414, 2020.

[29] A. Sabovic, M. Aernouts, D. Subotic, J. Fontaine, E. De Poorter, and J. Famaey, "Towards energy-aware tinyML on battery-less IoT devices," *Internet of Things*, vol. 22, p. 100736, 2023.

[30] A. Sabovic, M. Aernouts, D. Subotic, J. Fontaine, E. De Poorter, and J. Famaey, "Towards energy-aware tinyML on battery-less IoT devices," *Internet of Things*, vol. 22, p. 100736, 2023.

[31] A. Sabovic, J. Fontaine, E. De Poorter, and J. Famaey, "Energy-Aware TinyML Model Selection on Zero Energy Devices," *Internet of Things*, vol. 22, p. 100736, Dec. 2024.

[32] A. Sabovic, J. Fontaine, E. De Poorter, and J. Famaey, "Energy-Aware TinyML Model Selection on Zero Energy Devices," *Internet of Things*, 2024.

[33] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

[34] S. Somvanshi et al., "From Tiny Machine Learning to Tiny Deep Learning: A Survey," *arXiv preprint*, Texas State University, 2025.

[35] Swanson, A. B., Kosmala, M., Lintott, C. J., Simpson, R. J., Smith, A., & Packer, C. (2015). Snapshot Serengeti, high-frequency annotated camera trap images of 40 mammalian species in an African savanna. *Scientific Data*, 2, 150026.

[36] Z. Tang, Y. Sun, W. Chen, J. Ding, B. Ai, and Y. Shao, "Hierarchical Online-Scheduling for Energy-Efficient Split Inference with Progressive Transmission," *arXiv preprint arXiv:2409.08369*, 2024.

[37] van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI-16)*.

[38] M. Yi, X. Wang, J. Liu, Y. Zhang, and R. Hou, "Meta-Reinforcement Learning for Timely and Energy-efficient Data Collection in Solar-powered UAV-assisted IoT Networks," *arXiv preprint arXiv:2312.17081*, 2023.

[39] H. S. Zheng et al., "StreamNet: Memory-Efficient Streaming Tiny Deep Learning Inference on the Microcontroller," in *Proc. NeurIPS 2023*, 2023.